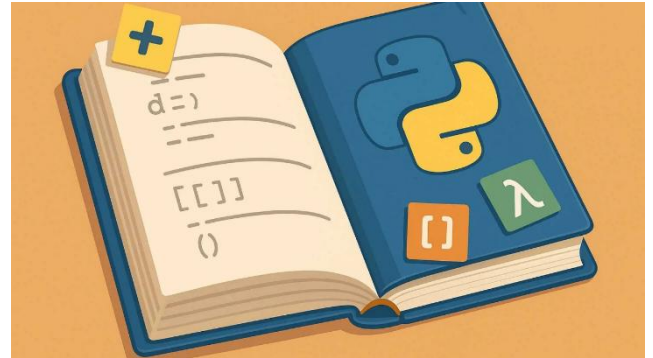


INTRODUCCIÓN A PYTHON PARA LA RESOLUCIÓN DE PROBLEMAS

Natalia R. Salaberry



DESCRIPCIÓN BREVE

Este libro tiene como propósito introducir al uso del lenguaje de programación Python para resolver diferente tipo de problemas. Cuenta con diversos ejemplos vinculados a distintas temáticas que se suelen enseñar en materias de las carreras en la Facultad de Ciencias Económicas. El fin último es facilitar un aprendizaje contextualizado desde el uso experimental.

Salaberry, Natalia Romina

Introducción a Python para la resolución de problemas / Natalia Romina Salaberry. -
1a ed. - Ciudad Autónoma de Buenos Aires : Universidad de Buenos Aires. Facultad de
Ciencias Económicas, 2026.

Libro digital, PDF

Archivo Digital: descarga

ISBN 978-950-29-2101-3

1. Algoritmo. I. Título.

CDD 005.1



AUTORIDADES

UNIVERSIDAD DE BUENOS AIRES

Rector: Dr. Ricardo J. Gelpi

FACULTAD DE CIENCIAS ECONÓMICAS

Decano: Prof. Emérito Dr. Ricardo J. M. Pahlen
Acuña

INSTITUTO DE INVESTIGACIONES EN ADMINISTRACIÓN, CONTABILIDAD Y MÉTODOS CUANTITATIVOS PARA LA GESTIÓN (IADCOM)

Director: Daniel A. Sarto

CENTRO DE INVESTIGACIÓN EN METODOLOGÍAS BÁSICAS Y APLICADAS A LA GESTIÓN (CIMBAGE, IADCOM)

Director: Dr. Javier I. García Fronti

Prólogo

En el entorno profesional y de investigación vinculados a las Ciencias Económicas, las planillas de cálculo fueron la herramienta fundamental para el manejo cuantitativo de datos. En los últimos años, la creciente complejidad de los datos, la necesidad de estimaciones más rigurosas y la búsqueda de modelos eficientes de predicción y clasificación han llevado a la necesidad de que los profesionales del área incorporen una lógica algorítmica de resolución de problemas.

Esta obra responde directamente a esa necesidad. Aprender a programar no es simplemente adquirir una competencia técnica; es incorporar una nueva estructura de pensamiento algorítmico, fundamental para la toma de decisiones en entornos organizacionales. Dentro de las herramientas disponible, el lenguaje *Python* se ha consolidado como el ecosistema predilecto en la economía, las finanzas y la gestión organizacional. Un aporte de este trabajo es su enfoque pedagógico, diseñado específicamente para la realidad y las materias del estudiante de nuestra Facultad de Ciencias Económicas de la Universidad de Buenos Aires. La autora, Natalia R. Salaberry, logra hacer entendible la lógica de programación, integrado entornos basados en la nube, como Google Colaboratory.

Desde el Centro de Investigación en Metodologías Básicas y Aplicadas a la Gestión (CIMBAGE), impulsamos estratégicamente la convergencia entre la innovación tecnológica, el análisis cuantitativo riguroso y la gobernanza de datos en el marco de la Inteligencia Artificial. Este texto representa un aporte sustancial a esa misión académica, sentando las bases para que los lectores puedan interpretar la realidad económica.

Invito a los y las estudiantes y colegas a sumergirse en estas páginas con una profunda vocación de descubrimiento. El recorrido que propone Natalia es un paso importante para la formación de un profesional que articule manejo de algoritmos con el pensamiento crítico propio de Ciencias Económicas.

Javier I. García Fronti

Director del CIMBAGE

Tabla de contenidos

Prólogo	3
Introducción	5
1. Nociones básicas de programación	6
1.1 Ejercicios propuestos	7
2. El lenguaje de programación Python	9
2.1 El entorno Google Colaboratory para la ejecución de Python	10
2.2 Tipo de datos de uso frecuente en Python	12
2.2.1 <i>Number</i>	12
2.2.2 <i>String</i>	14
2.2.3 <i>Boolean</i>	16
2.2.4 <i>Date</i>	18
2.2.5 <i>None, NaN y Zero</i>	20
2.3. Tipo de objetos más frecuentes en Python.....	20
2.3.1 <i>List</i>	21
2.3.2 <i>Tuple</i>	25
2.3.3 <i>Dictionary</i>	29
2.3.4 <i>Array</i>	33
2.3.5 <i>Matrix</i>	39
2.4. Ejercicios propuestos	46
3. Programación funcional con Python	48
3.1. Condicional IF	48
3.2. Bucle WHILE	51
3.3. Bucle FOR	55
3.4. Creación de Funciones	60
3.5. Ejercicios propuestos	65
4. Bibliografía	66
5. Anexo	66

Introducción

Durante muchos años, el uso de un lenguaje de programación se limitaba a una función específica del área informática, relacionada principalmente a tareas técnicas especializadas. Con el correr del tiempo, la masificación del uso de computadoras habilitó a profesionales de diversas disciplinas a poder automatizar tareas repetitivas como así también manejar mayores volúmenes de datos. Esto condujo a la necesidad de aprender a utilizar lenguajes de programación, pero ya no tanto con un perfil específicamente técnico sino más bien con un perfil analítico.

En este contexto, es posible preguntarse entonces porque puede resultar necesario aprender a usar un lenguaje de programación, e incluso programar funcionalmente, en el área de las Ciencias Económicas. Teniendo en cuenta que el estudiante, docente o profesional involucrado tiene o tendrá la necesidad de contar con información y gestionarla, la programación surge como facilitadora para llevar adelante dicha tarea. También, para brindar resolución a problemas mediante computadoras ya sea durante una etapa de formación o durante el ejercicio de la profesión. De aquí que se ha constituido en una habilidad complementaria a la vez que fundamentalmente necesaria, demandada en el mercado laboral.

Por lo aquí expuesto, el propósito del presente libro es introducir a la lógica programática mediante el uso del lenguaje de programación *Python*. Para adquirir la habilidad de uso de esta herramienta, será necesario poder comunicarse con una computadora para darle instrucciones adecuadas y así poder obtener el resultado deseado. Esto requerirá, en primer lugar, conocer la lógica abstracta del planteo de problemas para lograr instruir adecuadamente a una computadora. Indefectiblemente, lo anterior conduce a adquirir nociones generales de la programación. En segundo lugar, será necesario conocer el idioma propio del lenguaje, es decir, sus características generales y los tipos de objetos para saber cómo operar con él. Particularmente, se introduce al uso de *Python*, debido a que, en las últimas décadas, se ha posicionado como el predilecto en diversas áreas de la economía, por su versatilidad y facilidad de uso incluso para quien no es experto en la informática. Posteriormente, se podrá avanzar hacia su utilización en el contexto de resolución de problemas para diferentes áreas temáticas vinculadas a las Ciencias Económicas.

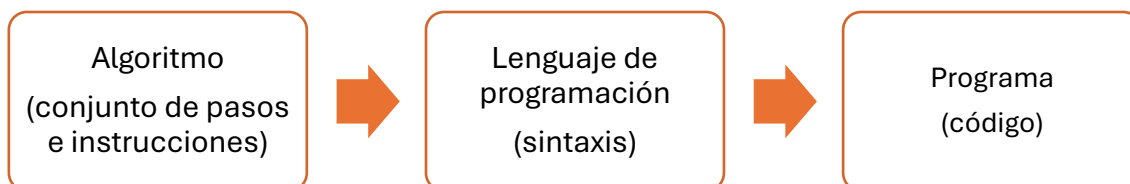
La propuesta de desarrollo del aprendizaje a lo largo del libro se basa en explicaciones conceptuales y características, pero también mediante ejemplos de uso concreto. Con ello, le lector podrá realizar un recorrido en etapas que se acompañan con cuadernos para la ejecución de códigos de manera online mediante links disponibles al finalizar cada apartado, por lo cual tan solo se requerirá de una conexión a internet para ejecutar *Python*. También, al final de cada capítulo encontrará un apartado con ejercicios propuestos de forma tal que pueda afianzar los conocimientos adquiridos. En la sección de anexo encontrará un *link* que lo dirige al repositorio online del libro donde encontrará, además de los cuadernos correspondientes a cada capítulo, el resolutorio de los ejercicios propuestos.

1. Nociones básicas de programación

En la actualidad, es sabido que una computadora es capaz de realizar cálculos a velocidades extraordinarias, superando ampliamente a la capacidad del ser humano para poder hacerlo. Ahora bien, a diferencia de este último, una computadora debe recibir instrucciones específicas (paso a paso) para que pueda brindar un resultado. En este sentido, podría decirse que requiere de un método imperativo de comunicación (Guttaj, 2017). Esto es, no solo requerirá de una instrucción directa (sumar, restar, entre otras) sino que también requiere de que se le especifique las condiciones bajo las cuales ejecutar la instrucción. Lo que resulta importante rescatar de lo anterior es que, mientras el ser humano puede comprender que debe hacer por sus propios medios y, en ocasiones, con información incompleta; las computadoras, requieren de instrucciones completas y detalladas.

Tal estructura específica requerida por las computadoras es lo que se conoce como **algoritmo**. Este refiere específicamente a una lista de instrucciones ordenadas y finitas – incluye todas las condiciones necesarias para poder operar– que describen un proceso de cálculo y que, al ejecutarse, producirá un resultado. En cambio, un **programa**, es el algoritmo convertido a código. Este último deberá realizarse en un idioma específico a través del cual la computadora podrá comprender las instrucciones dadas y que será ejecutado en memoria. Este idioma no es más que lo que se conoce como **lenguaje de programación** (Guttaj, 2017). Como sucede con cualquier idioma diferente al nativo, el lenguaje de programación tendrá su propio vocabulario y estructura de escritura (sintaxis).

Figura 1: Proceso de programación



Fuente: elaboración propia

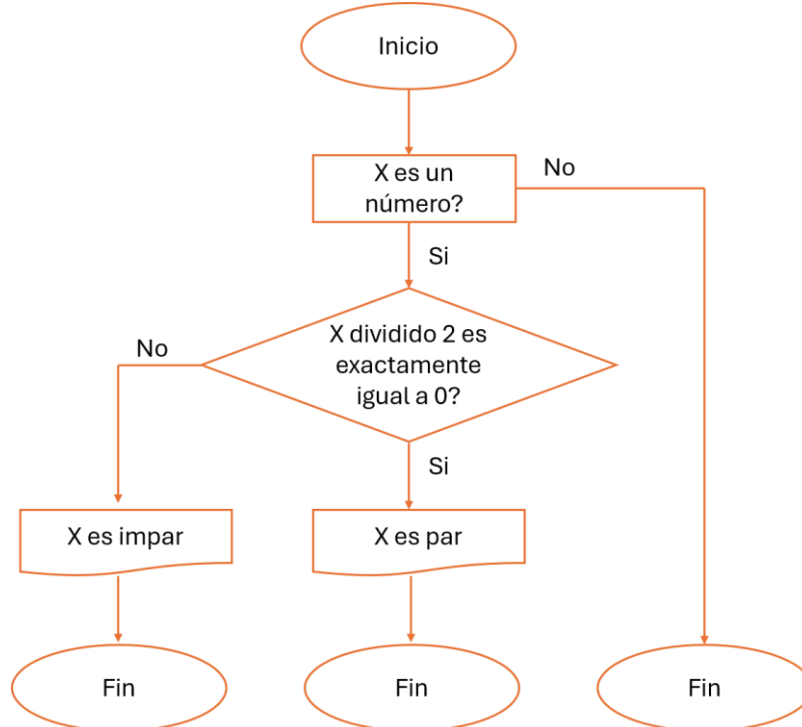
Ejemplo 1. Supóngase que se requiere que una computadora determine si un número (x) es par o impar. Como punto de partida se debe conocer cuáles son las condiciones que se deben cumplir para que un número sea par. Simplemente será que el número debe ser múltiplo de dos, es decir, el resto de dividir por dos debe ser cero. A partir de ello, es posible escribir el algoritmo:

- Iniciar con x
- Si el resto de dividir x por 2 es igual a 0, devolver “ x es par” y finalizar
- Si el resto de dividir x por 2 es distinto a 0, devolver “ x es impar” y finalizar
- Finalizar

Generalmente, para representar esquemáticamente un algoritmo, se utiliza un **diagrama de flujo** (Guttaj, 2017). Este consiste en un esquema gráfico donde se utilizan diferentes figuras para diferenciar Inicio y Fin (elipse), de pasos

(rectángulos), de condiciones a ser evaluadas (rombos), de orden de secuencia de ejecución (flechas), de resultados (Rectángulo con onda en la base). Para el algoritmo especificado en el Ejemplo 1, el diagrama de flujo es:

Figura 2: Diagrama de flujo para el ejemplo 1

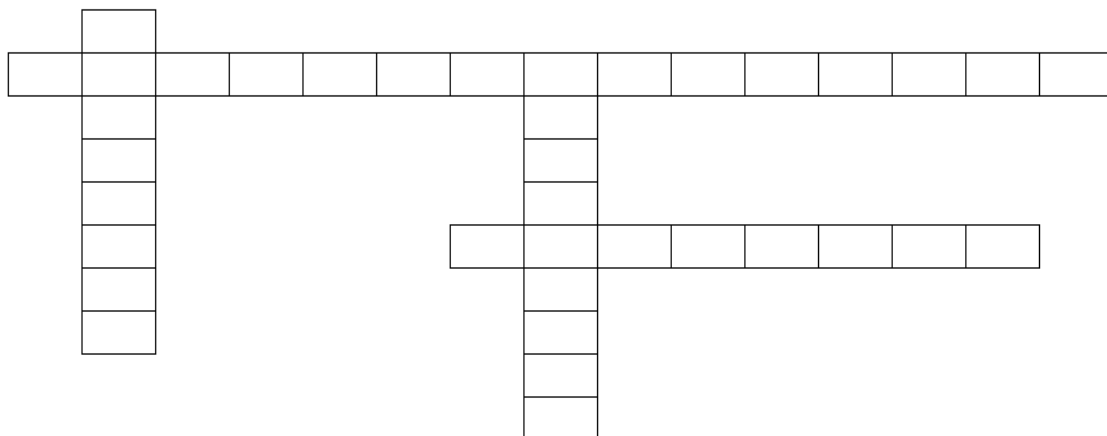


Fuente: elaboración propia

El siguiente paso consiste en poder plasmar el algoritmo en una sintaxis para convertirlo en un programa que pueda ser ejecutado en una computadora. Esto se retomará en el capítulo 3. Para lograrlo se deberá utilizar un lenguaje de programación. En el siguiente capítulo, se presenta el lenguaje de programación Python y sus principales particularidades.

1.1 Ejercicios propuestos

Ejercicio 1.1.1: complete el siguiente crucigrama a partir de las definiciones dadas



Definiciones horizontales

- Representación esquemática de un algoritmo
- Algoritmo convertido a código

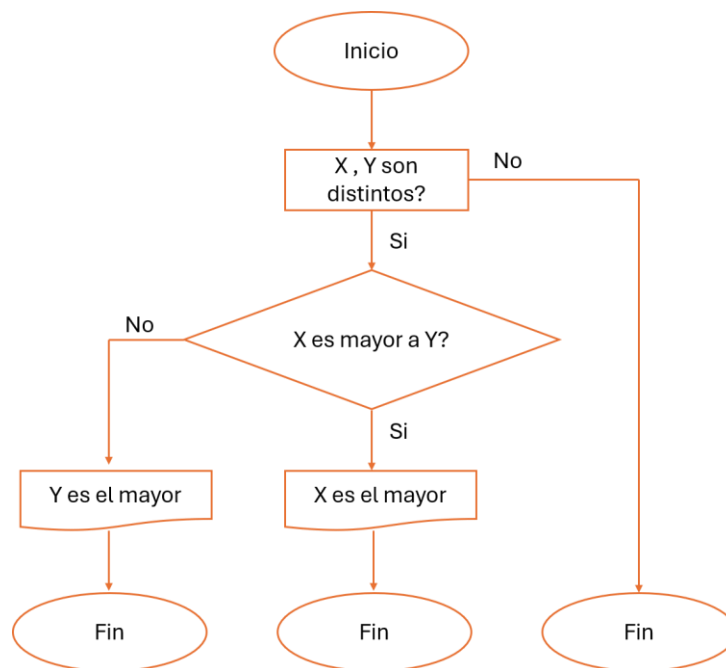
Definiciones verticales

- Lenguaje de programación
- Lista detallada de pasos e instrucciones

Ejercicio 1.1.2: dado el siguiente algoritmo para determinar si un año es bisiesto, realice el diagrama de flujo correspondiente.

- Iniciar con x
- Si x es entero, continuar
- Si x es mayor o igual que 1, continuar
- Si el resto de dividir x por 4 es igual a 0, continuar
- Si el resto de dividir x por 100 es distinto a 0, continuar
- Si el resto de dividir x por 400 es igual a 0, devolver “x es bisiesto” y finalizar
- Finalizar

Ejercicio 1.1.3: a partir del diagrama de flujo presentado a continuación, determine cual es el objetivo del problema a resolver y escriba el algoritmo correspondiente.



Ejercicio 1.1.4: dado el siguiente algoritmo para obtener la suma de los números comprendido entre 1 y 10, desarrolle el diagrama de flujo correspondiente.

- Iniciar
- Establecer numero=0 y suma=0, continuar
- Establecer numero=numero+1 y suma=suma+numero, continuar
- Si numero =10, devolver suma y finalizar; Si numero $\neq$ 10, volver al paso 3
- Finalizar

2. El lenguaje de programación Python

Python es un lenguaje de programación que fue creado en 1990 por Guido van Rossum en el centro Stichting Mathematisch de los Países Bajos. Presenta la característica de ser interpretado, razón por la cual puede ser ejecutado en cualquier sistema operativo y en cualquier plataforma. Esto debido a que posee su propio interprete (lenguaje) mediante el cual un código es ejecutado en tiempo real sin necesidad de compilarse previamente. De aquí que se trate de un lenguaje de alto nivel, es decir, con este se programa utilizando operaciones abstractas. Tal característica hace que *Python* pueda usarse para crear casi cualquier tipo de programa ya que no requiere acceso directo al hardware de la computadora, sino que opera en memoria (Guttaj, 2017).

En *Python*, todos los elementos son un **objeto** (Guttaj, 2017). Un número, un texto, una tabla de datos, entre otros, todos son objetos manipulables para *Python*. Cada uno de ellos tendrá un tipo asociado y a partir de ello el tipo de operaciones que se podrán realizar. Esto quiere decir que *Python* es un lenguaje de programación orientado a objetos razón por la cual es posible operar con estos de una manera sencilla.

Python es **open source**, es decir, tiene licencia libre y gratuita para su uso ya sea con fines comerciales o no. A su vez, esta particularidad ha generado una comunidad muy grande de usuarios a su alrededor a través de los años. No solo han contribuido a mejorar su desarrollo, sino que se ha creado amplia documentación y foros lo que brinda la posibilidad de contar con bastas referencias para consultas y soporte. Además, cuenta con su página web¹ propia donde es posible encontrar diversa documentación y libros para distintos niveles.

La misma gran comunidad de usuarios ha desarrollado **librerías**, siendo aquellas que contienen un conjunto de funciones específicas ya programadas y que se encuentran disponibles para su uso. Con esto es posible encontrar que *Python* cuenta con funciones programadas que son nativas y aquellas que no lo son, siendo estas últimas las pertenecientes a una librería. Para poder utilizar una librería, en primer lugar, se debe instalar el **paquete** (*package*). Esto se hará mediante la instrucción: `pip install [nombre del paquete]`. Una vez instalado, se deberá importar la librería (activarse todas las funciones en memoria) mediante la siguiente instrucción: `import [nombre de librería] as [alias]`. Generalmente las librerías se renombran mediante un alias para una escritura más sencilla en la posterioridad.

Hay muchas librerías desarrolladas, pero existen tres esenciales y fundamentales que abarcan las principales funcionalidades necesarias para la resolución de problemas y el análisis de datos. La primera librería es *NumPy*², cuyo nombre es una abreviación de *Numerical Python* (McKinney, 2018). Esta provee las funciones necesarias para realizar la mayoría de las aplicaciones científicas con datos numéricos. Entre estas se encuentran aquellas que permiten realizar operaciones

¹ Web de Python: <https://www.python.org/>

² Web de NumPy: <https://numpy.org/>

con matrices (*matrix*), vectores (*arrays*), distintas operaciones algebraicas, generación de números aleatorios, entre otros. Pensar y trabajar con estructuras matriciales mediante esta librería aporta a *Python* una eficiencia de procesamiento superior frente a otros lenguajes. Además, la lógica vectorial es la que permite transformar datos provenientes de formatos tabulares en vectores plausibles de ser operados por algoritmos (modelos predictivos programados) intermedios y avanzados, pertenecientes a la familia del aprendizaje automático (*Machine Learning*).

Instalación del paquete: `!pip install numpy`

Carga de librería: `import numpy as np`

La segunda librería es *Pandas*³, la cual provee multiplicidad de funcionalidades para trabajar con datos tabulares y series de tiempo (McKinney, 2018). Se tratan de funciones que permiten realizar la lectura de datos, su procesamiento, manipulación, limpieza y transformación. Todas estas tareas resultan necesarias y fundamentales en una etapa previa al análisis de datos final.

Instalación del paquete: `!pip install pandas`

Carga de librería: `import pandas as pd`

La tercera librería es *Matplotlib*⁴, la cual permite generar gráficos y otro tipo de visualizaciones (McKinney, 2018). Su creador, John D. Hunter, buscó combinar el diseño visual de MATLAB, con la posibilidad de manipular los atributos gráficos y la obtención de gráficos. Brinda la posibilidad de exportar los gráficos como imagen o visualizarlos en el mismo entorno de ejecución o embeberlos dentro de otra interfaz como ser una página web o un tablero de información. La flexibilidad que ofrece esta librería para realizar visualizaciones se convierte en fundamental para lograr llevar a cabo una comunicación más acertada de resultados.

Instalación del paquete: `!pip install matplotlib`

Carga de librería: `import matplotlib as plt`

Por todas las características mencionadas, es posible decir que *Python* es un lenguaje de fácil uso y muy flexible para crear casi cualquier cosa, una vez que se entiende su lógica operativa. Además, es el lenguaje con el cual se han generado todos los principales desarrollos del *Machine Learning* e incluso de Inteligencia Artificial. De aquí que se ha posicionado fuertemente en el mercado profesional, pero también en el ámbito académico desde hace varios años.

2.1 El entorno Google Colaboratory para la ejecución de Python

Existen diferentes entornos (IDE, *Integrated Development Environment*) para poder ejecutar *Python*. Se trata de una aplicación que permite tanto la escritura como la

³ Web de *Pandas*: <https://pandas.pydata.org/>

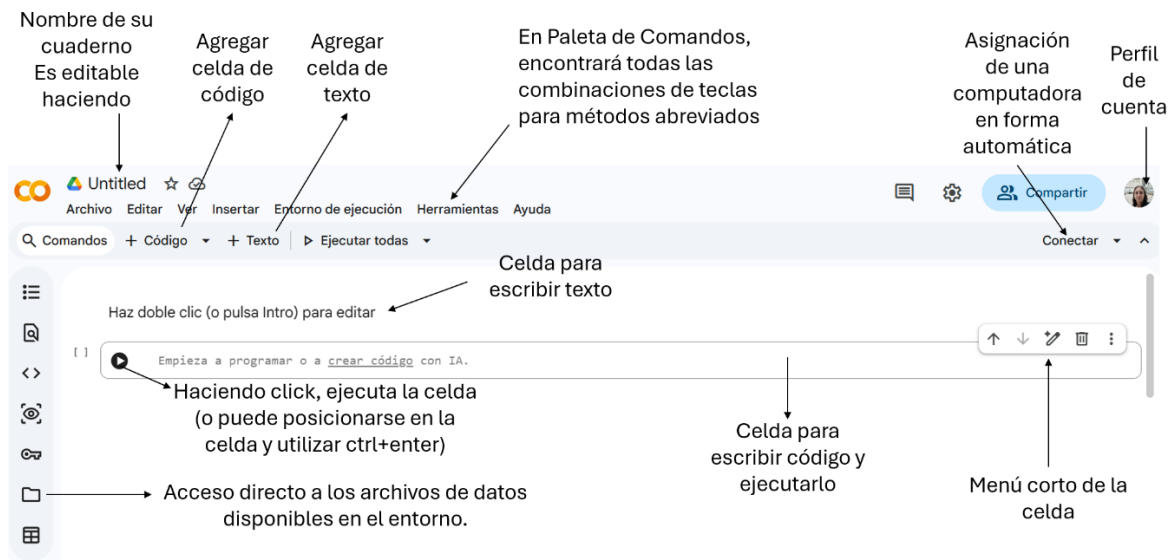
⁴ Web de *Matplotlib*: <https://matplotlib.org/>

ejecución del código de manera ordenada y simple, ya que se realiza en un solo lugar. En este libro se utilizará un entorno online denominado *Google Colaboratory*.

*Google Colaboratory*⁵, también llamado "*Colab*", permite ejecutar y programar en *Python* en tu navegador sin necesidad de instalar nada en tu computadora, así como tampoco contar con hardware avanzado. Solo se requiere una conexión a internet y contar con una cuenta de *Google*. Es un servicio de acceso libre, muy utilizado en ámbitos académicos. Entre las ventajas que posee se encuentra el hecho de que no se necesita realizar ninguna configuración; el servicio brinda alta capacidad de procesamiento en nube; es posible conectarse con su *Google Drive* personal facilitando de manera directa tanto el almacenamiento del *Colab* trabajado como de archivos de datos; permite compartir mediante *link* el contenido elaborado, entre otros.

A la estructura del entorno *Colab*, se lo denomina *Notebook* o cuaderno⁶. Esto porque su funcionalidad actúa como un cuaderno de notas interactivo, donde es posible combinar texto explicativo con código ejecutable. En las celdas de texto, es posible escribir ecuaciones o cualquier notación matemática utilizando *Latex*⁷, insertar imágenes, entre otros, ya que es un desarrollo *Mark Down*⁸. En las celdas de ejecución de código, también podrá escribir texto, pero anteponiendo el símbolo "#". Esto porque, en caso contrario, *Python* tratará de ejecutarlo y le dará error. En cambio, cuando se antepone el numeral, *Python* interpreta que eso no es un código ejecutable. Al ingresar a un cuaderno, se podrá observar lo siguiente:

Figura 3: Cuaderno *Colab*



Fuente: elaboración propia

El acceso a los cuadernos *Colab*, podrá ser de diferentes formas:

⁵ Más acerca de *Colab*: <https://colab.research.google.com/notebooks/welcome.ipynb?hl=es>

⁶ *Link* de acceso a un cuaderno *Colab*: <https://colab.research.google.com/drive/1JJgib-aGbFazltouPqVhM96zqkE2nsa?hl=es>

⁷ Acerca de *Latex*: <https://manualdelatex.com/simbolos>

⁸ Acerca de *Mark Down*: <https://www.markdownguide.org/>

- ✓ Si recibe un *link* de un cuaderno compartido solo con permiso de lectura, entonces deberá acceder al mismo y guardarlo. Para guardarlo ir a “Archivos” -> “Guardar una copia en Drive”. Al hacerlo se le abrirá el cuaderno en una nueva pestaña de su navegador y allí deberá conectarse y podrá ejecutar el código.
- ✓ Descargar un cuaderno para lo cual deberá ir a “Archivo” -> “Descargar”-> “Descargar .ipynb”. Obtendrá un archivo con extensión “.ipynb”. Para poder acceder al mismo deberá abrir un cuaderno nuevo y seleccionar “Archivo” -> “Abrir cuaderno”. En el buscador que se abrirá, deberá seleccionar el cuaderno descargado. Una vez cargado, conectarse y podrá ejecutar el código.
- ✓ En este libro, tendrá los cuadernos almacenados en un repositorio, de donde podrá descargarlos o directamente acceder a ellos haciendo clic en el botón “OpenInColab” que visualizará. En cada sección se le indicará un *link* de acceso.

Siempre que abra un cuaderno y se conecte, automáticamente se guardará en su *Drive*. Por lo tanto, si accede a este último, podrá ver los cuadernos trabajados. Si luego realiza modificaciones, podrá guardarlo desde “Archivo” -> “Guardar”.

2.2 Tipo de datos de uso frecuente en Python

Los principales tipos de datos que serán abordados a continuación son los *Numbers* (números), *String* (texto), *Boolean* (lógico) y *Date* (fecha). A su vez, se presentarán las principales funciones a través de las cuales es posible operar con estos en *Python*. De esta manera, se podrá, por un lado, conocer la lógica del lenguaje para poder comenzar a brindarle instrucciones; por el otro, comprender su manejo ya que luego podrán ser utilizados para resolver diferentes problemas o analizar datos.

2.2.1 Number

Los tres tipos de números de uso más frecuentes son los **enteros** (*Integers*), **decimales** (*Floating-point numbers*) y los **fraccionales** (*Fractions*). En *Python*, existe una **función** asociada a cada tipo de número que puede ser **nativa** o **depende de una librería**. Está permite definir un tipo específico sobre el dato, lo cual suele ser recurrente a la hora de dar una instrucción. En la tabla a continuación, se presentan los tipos de números mencionados y la función asociada:

Tabla 1: Tipo de números

Tipo de número	Ejemplo	Nombre en <i>Python</i>	Función asociada en <i>Python</i>	Requiere uso de librería
Entero	1	<i>Integer</i>	<code>int()</code>	No
Decimal	1.1	<i>Float</i>	<code>float()</code>	No
Fracción	$\frac{1}{2}$	<i>Fraction</i>	<code>Fraction()</code>	Si

Fuente: elaboración propia

Es importante notar que, por un lado, *Python* al ser un lenguaje de programación desarrollado en el idioma inglés, **el indicador de decimal es un punto** mientras que el separador de miles es una coma. Además, *Python* maneja una precisión de 16 decimales, a diferencia de una calculadora que solo muestra nueve.

Por otro lado, *Python* es **Case Sensitive**, es decir, sensible a mayúsculas y minúsculas. Por esta razón, para poder utilizar las funciones, se deberán escribir de forma exacta a como fueron creadas para que no se presente error. Por ejemplo, `Fraction()` inicia con mayúscula, en cambio `float()`, no.

Las operaciones aritméticas típicas que se utilizan con números son:

Tabla 2: Operaciones aritméticas típicas con números

Operación	Operador	Ejemplo de script	Output (resultado)	Requiere uso de librería
Suma	+	<code>1+1.1</code>	2.1	No
Resta	-	<code>4-2</code>	2	No
Multiplicación	*	<code>2*2</code>	4	No
División	/	<code>4/2</code>	2	No
Potencia	**	<code>2**2</code>	4	No
Raíz	**	<code>4**0.5</code>	2	No

Fuente: elaboración propia

Otra función nativa que resulta útil al operar con números es `round()`. Esta permite redondear un número, indicando la cantidad de decimales que se desea obtener. Por ejemplo:

```
round(1.2345, 2)
devuelve 1.23
```

```
round(1.235, 2)
```

devuelve 1.24

Para el área matemática, se encuentra la librería *math*⁹. Esta contiene casi todas las funciones matemáticas para realizar operaciones específicas. Entre estas se encuentran: factorial, combinatorio, logaritmo, el número Pi, entre muchas otras.

Ejemplo 2.2.1.1: resolver la siguiente operación:

$$\left(\frac{3.1 + \sqrt{4}}{\frac{1}{2} * 5} \right)^2$$

El código a ejecutar es:

```
from fractions import Fraction
((3.1 + (2**0.5)) / (Fraction(1,2)*5))**2
```

⁹ Funciones matemáticas de la librería math: <https://docs.python.org/es/3.10/library/math.html>

Cuyo resultado es 4.1616. Notar, que al igual que cuando se opera en una calculadora o escribiendo a mano, se deberán utilizar paréntesis para realizar cada operación en orden adecuado.

Ejemplo 2.2.1.2: cree un programa donde un usuario pueda ingresar el número de horas trabajadas y lo que cobra por hora. Como resultado se pretende obtener el pago que deberá recibir.

Para lograr la **interacción** con el usuario, existe la función nativa **input()** en *Python*. Con la entrada obtenida, el siguiente paso es asignarlo a un elemento para luego operar con ellos. Esto es lo que se conoce como **asignación** y consiste en crear un nombre seguido de un signo igual con el contenido que se desea. De este modo, se podrá realizar la operación con los elementos e imprimir el resultado.

El código para resolver el ejemplo 1, sería:

```
horas = float(input("Introduce tus horas de trabajo: "))
costo = float(input("Introduce lo que cobras por hora: "))
pago = horas * costo
print("El pago a recibir es $" + str(horas * costo))
```

Puede observarse que se define como *float()* al tipo de número ingresado debido a que, tanto las horas trabajadas como el valor hora, pueden ser un número decimal.

El resultado que se obtiene se verá del siguiente modo:

```
Introduce tus horas de trabajo: 10
Introduce lo que cobras por hora: 200
El pago a recibir es $2000.0
```

Nota: En el siguiente [LINK](#) podrá encontrar un cuaderno ejecutable (Sección 2.2.1 Numbers) donde se muestra cómo operar con los distintos tipos de números mencionados en esta sección a través de *Python*.

2.2.2 String

Los objetos *String* son una cadena de caracteres. Se trata de una cadena porque pueden ser una letra, una palabra, un párrafo que no solo contenga letras sino también símbolos particulares. Se definen entre comillas simples o dobles. Por ejemplo, si se define “hola”, *Python* entenderá que se trata de un objeto de tipo texto. También, si se define “123”. A pesar de observar un número, al estar definido entre comillas, *Python* lo interpretará como un literal, es decir, un texto.

En general, los *String* contienen varios elementos dado que cada letra o símbolo será un elemento. Por ello, se suelen asignar a un objeto el cual automáticamente será indexado. Esto es, *Python* creará un **índice implícito** que indicará la posición

donde se encuentra cada elemento. A diferencia de, por ejemplo, *Excel*, el índice en *Python* comienza en cero.

Supóngase que se tiene el *String* “HOLA!”. El índice creado automáticamente por *Python* será:

Índice implícito	0	1	2	3	4
<i>String</i>	H	O	L	A	!

A partir de ello, si, por ejemplo, se quiere obtener el elemento “L”, se deberá acceder a través de la posición asignada en el índice. En este caso, la posición será 2. En cambio, para la lógica interpretativa humana, el elemento “L” se encuentra en la posición 3, es decir, es la tercera letra. De aquí que, tener presente la lógica del índice en *Python*, resulta fundamental a la hora de poder operar con los distintos objetos.

Tomando como ejemplo el *String* asignado texto=“Ciencias Económicas”, en la tabla a continuación se muestran las operaciones típicas aplicables:

Tabla 3: Operaciones típicas con un *String*

Operación	Función	Ejemplo de script	Output (resultado)	Requiere uso de librería
Largo del objeto	len()	<i>len(texto)</i>	5	No
Convertir a mayúscula	upper()	<i>texto.upper()</i>	“CIENCIAS ECONÓMICAS”	No
Convertir a minúscula	lower()	<i>texto.lower()</i>	“ciencias económicas”	No
Separar el objeto	split()	<i>texto.split()</i>	[“Ciencias”, “Económicas”]	No
Concatenar	+	<i>texto + “ 2025”</i>	“Ciencias Económicas 2025”	No
Reemplazar	replace()	<i>texto.replace(“5”,“6”)</i>	“Ciencias Económicas 2026”	No

Fuente: elaboración propia

Ejemplo 2.2.2.1: A partir de la siguiente oración “La inflación de mayo 2026 es 2,1%” realice lo siguiente: cree un objeto *String* con el nombre texto que contenga a dicha oración. Separe el texto. Obtenga el elemento “2,1%” y asígnelo a un objeto denominado valor. Separe el objeto por %. Reemplace la coma por un punto. Obtengan el elemento 2.1. Conviértalo a número. Imprima el resultado.

El código a ejecutar es:

```
texto="La inflación de mayo 2026 es 2.1%"
texto=texto.split()
valor=texto[-1]
valor=valor.split("%")
valor=valor[0]
valor=valor.replace(",",".")
```



```
valor=float(valor)
print(valor)
```

de donde se obtiene: 2.1. Se puede observar que al definir un tipo *float* se obtiene como resultado un número y ya no un texto. A partir de ello, el objeto final valor podrá ser utilizado para realizar alguna operación aritmética de interés. Por otra parte, en el ejemplo anterior, se muestra que si se tiene un texto que contiene algún dato de interés se puede manipular para extraerlo.

Ejemplo 2.2.1.2: Escriba un programa que solicite al usuario que ingrese su primer nombre y apellido en minúscula. Conviértalo a mayúscula. A partir de ello, obtenga las iniciales. Finalmente, se pretende que el programa devuelva las iniciales del usuario.

El código a ejecutar es:

```
nombre=input('Ingresa su primer nombre y su apellido en minúscula: ')
nombre=nombre.upper()
nombre=nombre.split()
print('Las iniciales del usuario son: '+ nombre[0][0]+nombre[1][0])
```

Notar que el acceso a los elementos buscados requiere una doble indicación de posiciones. Esto porque, una vez separados los elementos, el objeto nombre contiene dos elementos: uno es el nombre propiamente dicho y otro el apellido del usuario. Luego, para obtener un elemento de nombre o de apellido, se debe indicar la posición del elemento dentro de este elemento individualizado. De aquí que, en realidad el objeto nombre, termina conformado por dos instancias de elementos.

El resultado que se obtiene se verá del siguiente modo:

```
Ingresa su primer nombre y su apellido en minúscula: juan lópez
Las iniciales del usuario son: JL
```

Nota: En el siguiente [LINK](#) podrá encontrar un cuaderno ejecutable (Sección 2.2.2 Strings) donde se muestra cómo operar con los Strings mencionados en esta sección a través de Python.

2.2.3 Boolean

Los objetos *Boolean* son un tipo particular que solo toma dos valores posibles: *true* (verdadero) o *false* (falso). Su uso frecuente es para validar una preposición o condición sobre cualquier tipo de objeto. Por ejemplo, si se escribe $3 > 2$, *Python* devolverá *True*. El símbolo mayor utilizado se conoce como **operador**. Existen varios operadores que permiten construir distintas condiciones de evaluación.

Tabla 4: Operadores

Operadores	Definición	Ejemplo de script	Output (resultado)	Requiere uso de librería
==	Exactamente igual	$a == b$	false	No
!=	Distinto que	$a != b$	true	No
>	Estrictamente mayor	$4 > 5$	false	No
<	Estrictamente menor	$2 < 3$	true	No
>=	Mayor o igual	$5 >= 4$	true	No
<=	Menor o igual	$3 <= 2$	false	No

Fuente: elaboración propia

Si un humano lee la palabra “casa” o “CASA”, conceptualmente interpreta de manera única. Es decir, el concepto no se diferencia por si está escrito en mayúscula o minúscula. ¿Qué sucede en *Python*? Como se mencionó en el apartado anterior, *Python* es *Case Sensitive*, razón por la cual interpretará que ambas palabras son diferentes. Si se escribe “casa”==“CASA” devolverá *False*. Esto permite comenzar a notar las particularidades de la lógica de un lenguaje computacional, lo que resulta importante para luego poder dar instrucciones de manera adecuadas a través de un *script*.

Ejemplo 2.2.3.1: indique verdadero o falso verificando con *Python*:

1. `"%"=="%"` -> True
2. `12.54 > 12.53` -> True
3. `1/2 >= 4/2` -> False

Ejemplo 2.2.3.2:

Escriba un programa donde el usuario ingrese una palabra dos veces: en minúscula y en mayúscula. Luego conviértalas a mayúscula. Se desea obtener como resultado si ambas palabras son iguales o no.

El código a ejecutar es:

```
palabra1=input('Ingrese una palabra en minúscula: ')
palabra2=input('Ingrese la misma palabra en mayúscula: ')
palabra1=palabra1.upper()
palabra2=palabra2.upper()
print('Las palabras ingresadas, ¿son iguales? '+str(palabra1==palabra2))
```

El resultado se verá como sigue:

```
Ingrese una palabra en minúscula: casa
Ingrese la misma palabra en mayúscula: CASA
Las palabras ingresadas, ¿son iguales? True
```

Nota: En el siguiente [LINK](#) podrá encontrar un cuaderno ejecutable (Sección 2.2.3 Booleans) donde se muestra cómo operar con los operadores mencionados en esta sección a través de *Python*.

2.2.4 Date

En Python, una fecha incluye el año, mes, día, hora, minuto, segundo, microsegundo y se denomina *DateTime*. Se trata de la representación de un instante temporal que es frecuente de ver en series de datos temporales como ser las económicas. Dado que es un desarrollo en el idioma inglés, se debe tener en cuenta que la fecha siempre será año, mes, día. Para poder crear este tipo de objeto y operar con él, se debe utilizar una librería denominada *datetime*.

```
from datetime import datetime
```

Para obtener la fecha actual, existe la función *now()*

```
datetime.now()
```

que devuelve:

```
datetime.datetime(2026, 6, 26, 20, 8, 7, 693409)
```

En general, las variables fecha son interpretadas como un *String* en Python. En dicho caso, se deberá convertir a tipo *DateTime*.

```
FECHA='2026-6-26 20:08:07'
```

El separador entre año, mes, día, en vez de un guion puede ser una barra, un espacio, entre otros. Se convierte a *DateTime*

```
DATETIME = datetime.strptime(FECHA, '%Y-%m-%d %H:%M:%S')
```

Al ser un tipo *DateTime*, *Python* lo interpreta internamente como un número y, en consecuencia, se podrán realizar sumas o restas de fechas.

Ejemplo 2.2.4.1: obtenga la cantidad de días transcurridos entre el 1/1/2025 y el 1/1/2026

```
DATETIME_1 = datetime.strptime('2026-1-1', '%Y-%m-%d')
DATETIME_2 = datetime.strptime('2025-1-1', '%Y-%m-%d')
DATETIME_1 - DATETIME_2
```

Se obtiene:

```
datetime.timedelta(days=365)
```

Por otra parte, *entre las* operaciones de uso frecuente, se encuentran el obtener parte de un *DateTime* como ser el año, el mes, el día o la hora, entre otros.

Tabla 5: Funciones asociadas a un *DateTime*

Acción	Función
Fecha Año, Mes, Día	date()
Año	year
Mes	month
Día	day

Número de día de la semana	weekday
Hora	hour
Minuto	minute
Segundo	second

Fuente: elaboración propia

También es posible convertir un *DateTime* a un formato más acotado, como se pudo observar en el ejemplo 2.2.4.1. O también, se puede extraer partes individuales de interés que, en general, se suelen utilizar para crear nuevas variables. Convirtiendo un *String* en *DateTime*, luego se podrán aplicar las funciones listadas a continuación.

Tabla 6: Funciones para extraer partes contenidas en una fecha

Acción	Función
Número de año acotado	strftime('%y')
Número de año completo	strftime('%Y')
Número de mes	strftime('%m')
Nombre del mes acotado	strftime('%b')
Nombre del mes completo	strftime('%B')
Nombre del día acotado	strftime('%a')
Nombre del día completo	strftime('%A')
Número del día en el año	strftime('%j')
Número de semana del año	strftime('%W')
Hora	strftime('%H')
Minuto	strftime('%M')
Segundo	strftime('%S')

Fuente: elaboración propia

Ejemplo 2.2.4.2: Escriba un programa donde el usuario ingrese su fecha de nacimiento en formato AÑO/MES/DIA y devuelva "La fecha de nacimiento es DIA NÚMERO de NOMBRE DEL MES EN ESPAÑOL de AÑO NÚMERO"

El código a ejecutar es:

```
FECHA=input('Ingresa la fecha en formato AÑO/MES/DIA: ')
FECHA=datetime.strptime(FECHA, '%Y/%m/%d')
DIA=FECHA.day
MESES={'January':'enero',
'February':'febrero', 'March':'marzo', 'April':'abril', 'May':'mayo', 'June':'junio', 'July':'julio', 'August':'agosto', 'September':'septiembre', 'October':'octubre', 'November':'noviembre', 'December':'diciembre'}
MES=MESES[FECHA.strftime('%B')]
AÑO=FECHA.strftime('%Y')
print('La fecha de nacimiento es '+str(DIA)+ ' de ' + MES + ' de ' + str(AÑO))
```

El resultado se verá como sigue:

Ingresa la fecha en formato AÑO/MES/DIA: 1950/4/25
La fecha de nacimiento es 25 de abril de 1950

Nota: En el siguiente [LINK](#) podrá encontrar un cuaderno ejecutable (Sección 2.2.4 Date) donde se muestra cómo operar con las fechas a través de Python.

2.2.5 None, NaN y Zero

En una buena parte de los lenguajes de programación existe el valor *NULL* (nulo), siendo un valor vacío. En *Python*, dicho valor se denomina *None* que representa la ausencia total de valor. Por ejemplo:

```
valor_1=None
valor_1
```

Verá que no se obtiene ningún resultado.

En cambio, *NaN* (*Not a Number*) es indicador de un dato numérico no válido o faltante. Por ejemplo, se define un tipo *float* para *nan*:

```
valor_2=float('nan')
valor_2
```

Devuelve *nan*. De aquí que no es un valor vacío, pero sí un valor no válido.

También, es un valor posible de obtener como resultado de operaciones matemáticas. Por ejemplo:

```
resultado=2/float('nan')
resultado
```

Devuelve *nan*. Aquí es posible notar que no devuelve un error, como sería el caso de dividir 2 por 0. Esto muestra que NaN es un valor diferente a cero.

Por otra parte, el valor *Zero* (cero) es el número 0, es decir, un número real.

```
valor_3=0
valor_3
```

Devuelve 0 siendo un valor válido.

O, por ejemplo, es un valor posible de obtener como resultado de una operación matemática:

```
resultado_3=10-10
resultado_3
```

Devuelve 0.

Nota: En el siguiente [LINK](#) podrá encontrar un cuaderno ejecutable (Sección 2.2.5 None, NaN y Zero) donde se muestra cómo operan los valores None, NaN y Zero a través de Python.

2.3. Tipo de objetos más frecuentes en Python

Los principales objetos que serán abordados en esta sección son *List* (lista), *Tuple* (tupla), *Dictionary* (diccionario), *Array* (vector) y *Matrix* (matriz). Cada uno de estos

se compondrá de elementos, lo cuales podrán contener los tipos de datos vistos en la sección 2.2. A su vez, se presentarán las principales funciones a través de las cuales es posible operar con estos en *Python*. De esta manera, se podrá comenzar a operar con objetos de mayor dimensión, lo que permitirá pensar en la resolución de problemas de mayor complejidad.

2.3.1 List

Las listas son una secuencia de elementos, donde estos últimos pueden ser de diferente tipo. En la sección 2.2.2, cuando se utilizó la función *split()* sobre un *String* se obtuvo como resultado ['Ciencias', 'Económicas']. Este “formato” de resultado, que se presenta entre corchetes y cada elemento está separado por una coma, es una lista. Además, las listas presentan la particularidad de ser mutables. Esto es, se puede modificar su contenido (los elementos), razón por la cual brindan flexibilidad a la hora de manipular el objeto.

A la vista del ojo humano, una lista tiene el formato de un vector, pero para Python no lo son. Esto porque una lista admite elementos de diferente tipo. En cambio, los vectores contienen elementos de un solo tipo. La importancia de manejar objetos de tipo listas es que se pueden asociar a cada columna de una tabla de datos.

Por ejemplo, se crea una lista:

```
lista=['uno', 'dos', 3,4]
```

Si se requiere observar el contenido del objeto creado, entonces:

```
lista
```

Devuelve

```
['uno', 'dos', 3, 4]
```

Como puede observarse, la lista creada tiene dos elementos tipo *String* ('uno', 'dos') y otros dos que son números enteros (3, 4)

Las listas tienen un índice implícito. Esto quiere decir, que para poder acceder a los elementos en ella contenidos, se deberá hacer seleccionando la posición según el índice.

Índice implícito	0	1	2	3
List[]	'uno'	'dos'	3	4

Por ejemplo, se quiere acceder al primer elemento:

```
lista[0]
```

Devuelve

```
'uno'
```

En el caso de que el elemento es un *String*, es posible acceder a cada elemento que lo compone. Por ejemplo, se quiere obtener la primera letra del primer elemento:

```
lista[0][0]
```

Devuelve

```
'u'
```

Esto quiere decir, que primero se debe acceder al elemento 'uno' y luego, también por posición de índice, al elemento 'u'.

También es posible acceder a más de un elemento de la lista a la vez. Por ejemplo:

```
lista[1:3]
```

Devuelve

```
['dos', 3]
```

Aquí es posible observar que, en el rango especificado para las posiciones, el valor 3 no está incluido, es decir, es un "hasta".

Como se mencionó al inicio, las listas son mutables. Esto quiere decir que es posible modificar los elementos que contienen. Para ello se deberá indicar la posición del o los elementos que se quieren modificar y asignar el o los valores correspondientes. Por ejemplo, para modificar un elemento individual:

```
lista[0]=1
```

Ahora en la lista se observará [1, 'dos', 3, 4]

O también, se puede modificar más de un elemento a la vez. Por ejemplo, si:

```
lista[0:2]=[1,2]
```

Ahora la lista quedará como [1, 2, 3, 4]. Observar que, en este último caso, la asignación de elementos reemplazantes debe ser en formato de lista. También hubiese sido posible de realizar lo anterior como `lista_1[:2]=[1,2]`. Se deberá tener en cuenta que esta opción solo es posible si el rango establecido para reemplazo comienza desde el elemento inicial en adelante.

Ahora bien, una lista puede contener elementos que sean cada uno de ellos otra lista. Esto se conoce como lista anidada. Por ejemplo, podemos crear tres listas `lista_1=[1,2,3]`, `lista_2=[4,5,6]`, `lista_3=[7,8,9]` y con estas crear una nueva lista:

```
lista_all=[lista_1, lista_2, lista_3]  
lista_all
```

Se observará que `lista_all` es `[[1, 2, 3], [4, 5, 6], [7, 8, 9]]`

La forma de acceso a los elementos de la lista y a los elementos contenidos en cada lista individualmente, se realiza de la misma forma que se mostró previamente para el caso de un *String*. Por ejemplo:

```
lista_all[1][0]
```

Devolverá el elemento 4

Las funciones más frecuentes de utilizar con listas se agrupan en dos tipos: las que no modifican la lista y las que sí. A continuación, se listan las funciones del primer caso:

Tabla 7: Funciones que no modifican una lista

Operación	Función	Ejemplo de script	Output (resultado)	Requiere uso de librería
Cantidad de elementos que contiene la lista	len()	<i>len(lista_all)</i>	3	No
Elemento de menor valor	min()	<i>min(lista_all)</i>	[1, 2, 3]	No
Elemento de mayor valor	max()	<i>max(lista_all)</i>	[7, 8, 9]	No
Sumar los elementos	sum()	<i>sum(lista)</i>	10	No
Sumar los elementos contenidos en otro elemento	sum()	<i>sum(lista_all[0])</i>	6	No
Buscar un elemento dentro de la lista	in	[1,2,3] in lista_all	True	No
Buscar la posición de un elemento	index()	lista_all.index([1,2,3])	0	No
Contar las veces que aparece un elemento	count()	lista_all.count([1,2,3])	1	No
Verificar si todos los elementos de la lista son válidos	all()	all(lista_all)	True	No
Verificar si alguno de los elementos de la lista es válidos	any()	any(lista_all)	True	No
Concatenar un elemento	+	lista_all + ['item', 'item2']	[[1, 2, 3], [4, 5, 6], [7, 8, 9], 'item', 'item2']	No

Fuente: elaboración propia

Entre las funciones más frecuentes para modificar una lista, se encuentran:

Tabla 8: Funciones que modifican una lista

Operación	Función	Ejemplo de script	Output (resultado)	Requiere uso de librería
Añadir un elemento al final de la lista	<code>append()</code>	<code>lista.append(5)</code>	[1, 2, 3, 4, 5]	No
Añadir los elementos de una lista a otra	<code>extend()</code>	<code>lista_1.extend(lista_2)</code>	[1, 2, 3, 4, 5, 6]	No
Insertar un valor (vacío o no) en una lista	<code>insert()</code>	<code>lista_1.insert(4, None)</code>	[1, 2, 3, 4, None, 5, 6]	No
Eliminar un elemento de la lista, indicando cual	<code>remove()</code>	<code>lista_1.remove(None)</code>	[1, 2, 3, 4, 5, 6]	No
Eliminar un elemento de la lista por posición	<code>pop()</code>	<code>lista_1.pop(4)</code>	[1, 2, 3, 4, 6]	No
Invertir el orden de los elementos	<code>reverse()</code>	<code>lista_2.reverse()</code>	[6, 5, 4]	No
Ordenar los elementos de menor a mayor	<code>sort()</code>	<code>lista_2.sort()</code>	[4, 5, 6]	No

Fuente: elaboración propia

Ejemplo 2.3.1.1: A partir de que un usuario ingrese los valores 1,2,3, cree una lista con ellos. Utilizando la función `append()` agregue el valor 4 y 4+1 a la lista. Finalmente, que se obtenga "La suma de los elementos de la lista es: suma", donde suma debe mostrar el resultado de sumar los elementos.

El código a ejecutar es:

```
lista=[int(input('Ingrese el valor 1: ')),int(input('Ingrese el
valor 2: ')),int(input('Ingrese el valor 3: '))]
lista.append(4)
lista.append(4+1)
suma=sum(lista)
print('La suma de los elementos de la lista es: '+str(suma))
```

El resultado se verá como sigue:

```
Ingrese el valor 1: 1
Ingrese el valor 2: 2
Ingrese el valor 3: 3
La suma de los elementos de la lista es: 15
```

Ejemplo 2.3.1.2: Cree una lista con los precios 100,20,30. Cree otra lista con los valores computadora, teclado, software, cuyos precios correspondientes son los de la primera lista (en ese orden). Solicite al usuario el producto sobre el cual le interesa obtener el precio. Utilícelo para obtener el índice correspondiente y que se muestre por pantalla el siguiente resultado "El precio del producto X es \$P "

El código a ejecutar es:

```
precios=[100,20,30]
productos=['computadora', 'teclado', 'software']
pos=productos.index(input('Indique el producto en minúscula: '))
prod_elegido=productos[pos]
precio=precios[pos]
print('El precio del producto ' + str(prod_elegido) + ' es $ ' + str(precio))
```

Si se elige el producto computadora, el resultado se verá como sigue:

```
Indique el producto en minúscula: computadora
El precio del producto computadora es $ 100
```

Nota: En el siguiente [LINK](#) podrá encontrar un cuaderno ejecutable (Sección 2.3.1 List) donde se muestra cómo operar con las listas a través de Python.

2.3.2 Tuple

Las tuplas son una secuencia de elementos al igual que las listas, donde estos últimos pueden ser de diferente tipo. A diferencia de las listas, las tuplas presentan la particularidad de no ser mutables. Esto es, no se puede modificar ninguno de los elementos en ella contenidos. Sin duda esto genera trabajar con un objeto rígido o inflexible, pero muchas veces resulta adecuado. De aquí que, si un programa requiere utilizar una secuencia de elementos que no puede ni debe alterarse en el tiempo, entonces el tipo de objeto adecuado a definir es una tupla. Por ejemplo, tener la secuencia de nombre de provincias argentinas para luego operar con esta; o listado el abecedario, entre muchos otros.

El lugar más frecuente donde se observan tuplas es en las funciones ya programadas o aquellas que desarrolle el usuario. Esto es, cuando se utiliza una función, se debe citar () y dentro de estos deben ponerse los argumentos necesarios. Por ejemplo, cuando en el caso de las listas se utilizó *index()*, observará que se especificó con paréntesis y dentro de estos se indicó el argumento. Dicho formato de especificación es porque los valores de parámetros dados a una función son inmutables mientras se ejecuta la misma, al igual que en lógica matemática. Esto resulta muy importante de comprender y quedará plasmado cuando se vea creación de funciones en el capítulo 3.

La forma típica de crear una tupla es:

```
tupla=(1,2,'tres')
```

Si se requiere observar el contenido del objeto creado, entonces:

```
tupla
```

Devuelve

```
(1, 2, 'tres')
```

Como puede observarse, la tupla creada tiene dos elementos tipo `Number` (1, 2) y un elemento tipo `String` ('tres').

Otra forma de crear tuplas es:

```
tupla_1=1,2,3
tupla_1
```

Devuelve

```
(1, 2, 3)
```

Es posible observar que el resultado se muestra entre paréntesis, razón por la cual puede concluirse que el objeto creado se trata de una tupla.

Las tuplas tienen un índice implícito. Esto quiere decir, que para poder acceder a los elementos en ella contenidos, se deberá hacer seleccionando la posición según el índice, del mismo modo visto para el caso de las listas en el apartado anterior.

Índice implícito	0	1	2
<i>tupla ()</i>	1	2	'tres'

Como se mencionó al inicio, las tuplas son inmutables. Esto quiere decir que no posible modificar los elementos que contienen. Por ejemplo, si se intenta modificar un elemento individual:

```
tupla[0]='a'
```

Devuelve

```
TypeError: 'tuple' object does not support item assignment
```

De aquí que, el propio lenguaje, le advierte que el objeto tupla no admite asignación de elementos. Esto es, no es posible reemplazar algún elemento ya que no es mutable. Esta característica de inmutabilidad, tampoco, le permitirá agregar elementos mediante una función que modifica elementos como ser *append()*. Lo que, si es posible, utilizar la concatenación y el resultado asignarlo a un nuevo objeto:

```
tupla_2=tupla + ('cuatro',)
tupla_2
```

Devuelve (1, 2, 'tres', 'cuatro')

Al igual que las listas, una tupla puede contener elementos que sean cada uno de ellos otra tupla. Es decir, se pueden crear tuplas anidadas. Por ejemplo, podemos crear tres listas *tupla_A=(1,2,3)*, *tupla_B=(4,5,6)*, *tupla_C=(7,8,9)* y con estas crear una nueva tupla:

```
lista_all=[lista_1, lista_2, lista_3]
lista_all
```

Se observará que `tupla_all` es `((1, 2, 3), (4, 5, 6), (7, 8, 9))`

La forma de acceso a los elementos de la tupla y a los elementos contenidos en cada tupla individualmente, se realiza de la misma forma que se mostró en el apartado anterior para las listas anidadas.

Las funciones más frecuentes posibles de utilizar con tuplas son aquellas que no modifican el objeto original tupla. En consecuencia, serán las mismas funciones contenidas en la tabla 7 del apartado 2.3.1.

Ahora bien, la flexibilidad de operar con *Python* sigue estando presente desde el momento en que este tipo de objetos puede convertirse en lista. Por ejemplo:

```
LISTA=list(tupla)
LISTA
```

Se obtiene `[1, 2, 'tres']`. A partir de aquí, podrá utilizar el nuevo objeto lista mediante todas las operaciones vistas en el apartado anterior.

También es posible convertir una lista en una tupla:

```
TUPLA=tuple(LISTA)
TUPLA
```

Se obtiene `(1, 2, 'tres')`.

Ejemplo 2.3.2.1: Dada la siguiente tupla que contiene las provincias argentinas (Ciudad Autónoma de Buenos Aires, Provincia de Buenos Aires, Catamarca, La Pampa, Río Negro, Santa Cruz, Tucumán, Chaco, Formosa, Misiones, San Juan, Tierra del Fuego, Corrientes, La Rioja, Neuquén, Santiago del Estero, Córdoba, Mendoza, Santa Fe, Chubut, Jujuy, San Luis, Entre Ríos, Salta), obtener una nueva tupla pero ordenada alfabéticamente.

El código a ejecutar es:

```
tupla_original=('Ciudad Autónoma de Buenos Aires', 'Provincia de
Buenos Aires', 'Catamarca', 'La Pampa',
               'Río Negro', 'Santa Cruz', 'Tucumán', 'Chaco',
               'Formosa', 'Misiones', 'San Juan', 'Tierra del Fuego',
               'Corrientes', 'La Rioja', 'Neuquén', 'Santiago del
Estero', 'Córdoba', 'Mendoza', 'Santa Fe', 'Chubut',
               'Jujuy', 'San Luis', 'Entre Ríos', 'Salta')
lista=list(tupla_original)
lista.sort()
tupla_ordenada=tuple(lista)
tupla_ordenada
```

El resultado se verá como sigue:

```
(
    'Catamarca',
    'Chaco',
    'Chubut',
    'Ciudad Autónoma de Buenos Aires',
    'Corrientes',
    'Córdoba',
    'Entre Ríos',
    'Formosa',
    'Jujuy',
    'La Pampa',
    'La Rioja',
    'Mendoza',
    'Misiones',
    'Neuquén',
    'Provincia de Buenos Aires',
    'Río Negro',
    'Salta',
    'San Juan',
    'San Luis',
    'Santa Cruz',
    'Santa Fe',
    'Santiago del Estero',
    'Tierra del Fuego',
    'Tucumán')

```

Ejemplo 2.3.1.2: Cree un programa donde, a partir de un inventario existente ('A1','A2','B3','B4','C5','C6','D7','D8','D9','D10'), el usuario ingrese un código de producto y devuelva True si el producto existe en inventario o False en caso contrario

El código a ejecutar es:

```
prod_usuario=input('Ingrese el código de producto: ')
prod_inventario=('A1','A2','B3','B4','C5','C6','D7','D8','D9','D10')
resultado=prod_usuario in prod_inventario
print('El producto ' + str(prod_usuario) + ' existe en inventario ' +
      str(resultado))

```

Si se elige el producto A1, el resultado se verá como sigue:

```
Ingrese el código de producto: A1
El producto A1 existe en inventario True

```

Si se elige el producto C10, el resultado se verá como sigue:

```
Ingrese el código de producto: C10
El producto C10 existe en inventario False

```

Nota: En el siguiente [LINK](#) podrá encontrar un cuaderno ejecutable (Sección 2.3.2 Tule) donde se muestra cómo operar con las tuplas a través de Python.

2.3.3 Dictionary

Los diccionarios son un objeto que permite asociar una clave a un valor, donde estos últimos pueden ser datos de diferente tipo u objetos. Junto con las listas, son los tipos de objetos que se presentarán como más frecuentes para su uso a la hora de manipular datos para su análisis. Por ejemplo, mediante un diccionario se pueden modificar valores, asignar valores nuevos como ser el caso cuando se quiere recategorizar una variable, entre otros.

Como todo objeto en *Python* posee un índice, pero ahora este vendrá dado por la clave. Las claves, tiene que ser valores únicos y puede ser un *String* o un número. Al igual que las listas, son mutables, es decir, podrá modificarse los elementos contenidos después de creados. Para su creación deberá utilizarse llaves. Por ejemplo, se crea un diccionario:

```
diccionario = {'nombre': 'Rosa', 'apellido': 'Perez', 'año_nacimiento': 1983}
```

Si se requiere observar el contenido del objeto creado, entonces:

```
diccionario
```

Devuelve

```
{'nombre': 'Rosa', 'apellido': 'Perez', 'año_nacimiento': 1983}
```

Como puede observarse, el diccionario creado tiene dos elementos tipo *String* ('Rosa', 'Perez') y otro que es un número entero (1983)

También es posible crear diccionarios a partir de listas o tuplas. Por ejemplo:

Con listas

```
lista1=['A','B']
lista2=[1,2]
diccionario1=dict(zip(lista1,lista2))
diccionario1
```

Devuelve: {'A': 1, 'B': 2}

Con tuplas

```
lista1=('A','B')
lista2=(1,2)
diccionario2=dict(zip(lista1,lista2))
diccionario2
```

Devuelve: {'A': 1, 'B': 2}

El índice ahora vendrá dado por las claves. Esto quiere decir que, para poder acceder a los elementos contenidos, se deberá hacer seleccionando el nombre de la clave.

Índice	'nombre'	'apellido'	'año_nacimiento'
<i>Dictionary {}</i>	'Rosa'	'Perez'	1983

Por ejemplo, se quiere acceder al primer elemento:

```
diccionario['nombre']
```

Devuelve: 'Rosa'

En el caso de que el elemento sea un *String*, es posible acceder a cada elemento que lo compone. Por ejemplo, se quiere obtener la primera letra del primer elemento:

```
diccionario['nombre'][0]
```

Devuelve: 'R'

Esto quiere decir, que primero se debe acceder al elemento 'nombre' y luego, por posición de índice (implícito), al elemento 'R'.

Como se mencionó al inicio, los diccionarios son mutables. Esto quiere decir que es posible modificarlo. Para ello se deberá indicar la clave del elemento que se quiere modificar y asignar el o los valores correspondientes. Por ejemplo, para modificar un elemento individual:

```
diccionario['nombre']='Juan'
```

Ahora se observará que devuelve: {'nombre': 'Juan', 'apellido': 'Perez', 'año_nacimiento': 1983}

También es posible agregar una tupla clave valor. Por ejemplo:

```
diccionario['nacionalidad']='Argentina'
```

Ahora el diccionario contiene:

```
{'nombre': 'Juan',
 'apellido': 'Perez',
 'año_nacimiento': 1983,
 'nacionalidad': 'Argentina'}
```

Las funciones más frecuentes de utilizar con diccionarios se agrupan en dos tipos: las que no modifican un diccionario y las que sí. A continuación, se listan las funciones del primer caso:

Tabla 9: Funciones que no modifican un diccionario

Operación	Función	Ejemplo de script	Output (resultado)	Requiere uso de librería
Cantidad de elementos	len()	len(diccionario)	3	No

Clave de menor valor	min()	<i>min</i> (diccionario)	apellido	No
Clave de mayor valor	max()	<i>max</i> (diccionario)	nombre	No
Sumar los elementos (si son números)	sum()	<i>sum</i> (diccionario_2)	6	No
Sumar los elementos contenidos en otro elemento	sum()	<i>sum</i> (diccionario_2['C'])	6	No
Buscar una clave	in	'nombre' in diccionario	True	No
Buscar un valor	in	'Juan' in diccionario.values()	True	No
Obtener las claves	keys()	diccionario.keys()	dict_keys(['nombre', 'apellido', 'año_nacimiento'])	No
Obtener los valores	values()	diccionario.values()	dict_values(['Juan', 'Perez', 1983])	No
Obtener los elementos	items()	diccionario.items()	dict_items([('nombre', 'Juan'), ('apellido', 'Perez'), ('año_nacimiento', 1983)])	No
obtener un elemento a partir de la clave	get()	diccionario.get('nombre')	'Juan'	No
Vaciar un diccionario	clear()	Diccionario_2.clear()	{}	No

Fuente: elaboración propia

Entre las funciones más frecuentes que modifican un diccionario, se encuentran:

Tabla 10: Funciones que modifican un diccionario

Operación	Función	Ejemplo de script	Output (resultado)	Requiere uso de librería
Añadir un elemento	=	<i>diccionario['nacionalidad']='Argentina'</i>	{'nombre': 'Juan', 'apellido': 'Perez', 'año_nacimiento': 1983, 'nacionalidad': 'Argentina'}	No
Añadir los elementos de un diccionario en otro	update()	<i>d1={'A':1,'B':2} d2={'C':3,'D':4} d1.update(d2)</i>	{'A': 1, 'B': 2, 'C': 3, 'D': 4}	No
Eliminar elementos por clave	pop()	<i>d1.pop('D')</i>	'A': 1, 'B': 2, 'C': 3}	No

Eliminar elementos por clave	del	<i>del d1['B']</i>	{'A': 1}	No
Eliminar la última tupla clave valor	popitem()	<i>d1.popitem()</i>	{'A': 1, 'B': 2}	No

Fuente: elaboración propia

Ejemplo 2.3.3.1: Escribir un programa que guarde en un objeto el siguiente diccionario {'Euro':'€', 'Peso':'\$', 'Yen':'¥'}. Luego que el usuario ingrese el nombre de una divisa y devuelva su símbolo. En caso de que no esté en el diccionario, que muestre un mensaje de aviso.

El código por ejecutar es:

```
monedas = {'Euro':'€', 'Peso':'$', 'Yen':'¥'}
moneda = input("Introduce el nombre de una divisa: ")
print('El símbolo correspondiente a ' + moneda + ' es: ' +
monedas.get(moneda.title(), "La moneda elegida no está."))
```

El resultado se verá como sigue:

```
Introduce el nombre de una divisa: peso
El símbolo correspondiente a peso es: $
```

Si en cambio, el usuario ingresa dólar:

```
Introduce una divisa: dólar
El símbolo correspondiente a dólar es: La moneda elegida no está.
```

Ejemplo 2.3.3.2: Cree un programa donde el usuario introduzca 3 productos y el precio de cada uno y usted conforme un diccionario. Agregue el subtotal gastado. Luego al subtotal aplique un descuento del 15% y agréguelo al diccionario. Al final que el programa devuelva cada ítem con su precio, el subtotal, el monto del descuento y el total pagado.

El código a ejecutar es:

```
compra={}
item1=input('Nombre del producto1: ')
precio1=input('Precio del producto1: ')
item2=input('Nombre del producto2: ')
precio2=input('Precio del producto2: ')
item3=input('Nombre del producto3: ')
precio3=input('Precio del producto3: ')
compra[item1]=float(precio1)
compra[item2]=float(precio2)
compra[item3]=float(precio3)
compra['Subtotal']=compra.get(item1)+compra.get(item2)+compra.get(item3)
compra['Descuento']=compra['Subtotal']*0.15
compra['Total']=compra['Subtotal']-compra['Descuento']
print('El cliente compró:')
print(list(compra.keys())[0], compra.get(item1))
```

```

print(list(compra.keys())[1], compra.get(item2))
print(list(compra.keys())[2], compra.get(item3))
print(list(compra.keys())[3], compra.get('Subtotal'))
print('Se aplicó un descuento del 15%: ' +
      str(compra.get('Descuento')))
print('En total pagó:' + str(compra.get('Total')))

```

El resultado es:

```

Nombre del producto1: computadora
Precio del producto1: 1500000
Nombre del producto2: teclado
Precio del producto2: 70000
Nombre del producto3: mouse
Precio del producto3: 30000
El cliente compró:
computadora 1500000.0
teclado 30000.0
mouse 30000.0
Subtotal 1560000.0
Se aplicó un descuento del 15%: 234000.0
En total pagó:1326000.0

```

Nota: En el siguiente [LINK](#) podrá encontrar un cuaderno ejecutable (Sección 2.3.3 Dictionary) donde se muestra cómo operar con los diccionarios a través de Python.

2.3.4 Array

Un arreglo (array) es un conjunto de elementos ordenados, donde estos últimos deben ser del mismo tipo. Presentan la característica de ser mutables, al igual que las listas. Para poder definir y operar con un array en *Python*, se deberá utilizar la librería *numpy*, la cual fue presentada al comienzo del presente capítulo. Esto quiere decir, que primero deberá importarse la librería y luego proceder con la creación del objeto. Por ejemplo:

```
import numpy as np
```

Se crea un objeto array:

```
vector= np.array([1,2,3,4])
vector
```

Devuelve: array([1, 2, 3, 4])

También es posible crear un *array* a partir de una lista o tupla. Por ejemplo:

```
lista=[1,2]
vector_1=np.array(lista)
vector_1
```

Devuelve: `array([1, 2])`

```
tupla=(1,2)
vector_t=np.array(tupla)
vector_t
```

Devuelve: `array([1, 2])`

La noción de *array* se asocia a la de vector el cual se define como una n-tupla siendo un punto en un espacio de dimensión n (Chiang y Wainwright, 2006). Un vector posee dos características definitorias: es atómico, es decir, puede contener elementos de un solo tipo; y tiene dimensión única que viene dada por el largo (cantidad de elementos contenidos) del vector. Su representación es como sigue:

$$v = [a_1, a_2, \dots, a_n] \quad v \in \mathbb{R}^n$$

donde v es un vector fila de dimensión $1 \times n$. Si en cambio:

$$v' = [a_1, a_2, \dots, a_n] \quad v' \in \mathbb{R}^n$$

v' se trata de un vector columna cuya dimensión es $n \times 1$

Esta definición algebraica, proveniente de la ciencia matemática, es interpretada del mismo modo en *Python*. Si se quiere crear un vector fila, entonces:

```
vector_1=np.array([1,2,3,4])
```

Devuelve: `array([1, 2, 3, 4])`

O se puede crear un vector columna del siguiente modo:

```
vector_2=np.array([1],[2],[3],[4])
```

Devuelve: `array([[1],
[2],
[3],
[4]])`

Como puede observarse, el `vector_1` (fila) posee dimensión 1×4 mientras que el `vector_2` (columna) posee dimensión 4×1 . A la hora de hacer operaciones entre vectores, será importante tener en cuenta las dimensiones.

Continuando con el objeto *array*, si, por ejemplo, se quiere crear un *array* con números del 1 al 10, se puede utilizar la función *arange()* de la librería *numpy()*:

```
np.arange(1, 11)
```

Devuelve: `array([ 1, 2, 3, 4, 5, 6, 7, 8, 9, 10])`

Notar que el valor del primer argumento de la función *arange()* es un “inclusive”, mientras que el segundo argumento es un “hasta”.

O si, por ejemplo, se quiere crear un *array* con números del 1 al 10, pero que los elementos vayan de dos en dos, entonces:

```
np.arange(1, 11, 2)
```

Devuelve: `array([1, 3, 5, 7, 9])`

Al igual que las listas, un *array* tiene un índice implícito en *Python*. Esto quiere decir que, para obtener los elementos del objeto de manera individualizada, se debe citar la posición de cada uno.

Índice implícito	0	1	2	3
<code>array()</code>	1	2	3	4

Por ejemplo, se obtiene el primer elemento del *array* creado inicialmente:

```
vector[0]
```

Devuelve: `np.int64(1)`

Como se mencionó, un *array* es mutable, razón por la cual es posible modificarlo o modificar sus elementos. En este último caso, se deberá indicar la posición del elemento que se quiere modificar y asignar el valor correspondiente. Por ejemplo, para modificar un elemento individual:

```
vector[0]=0
```

Ahora se observará que devuelve: `array([0, 2, 3, 4])`

También es posible reemplazar de a varios elementos a la vez:

```
vector[0:2]=[1,2]
```

Ahora se observará que devuelve: `array([1, 2, 3, 4])`

Para agregar un nuevo elemento, se deberá utilizar la función *append()*, presentada en la sección de listas, pero ahora dependiente de la librería *numpy*:

```
vector=np.append(vector,5)
```

Como puede observarse, el *array* contiene también al elemento 5: `array([1, 2, 3, 4, 5])`. Aquí es importante aclarar que el objeto *array* original se modificó porque fue asignado el resultado obtenido al mismo objeto original. Si no se realiza la asignación, el objeto original no quedará modificado, solamente se observará un resultado que incluye el elemento 5.

Las funciones más frecuentes de utilizar con *array* se agrupan en dos tipos: las que no modifican un *array* y las que sí. En este último caso, *tener en cuenta que se debe asignar el resultado para tener una modificación real del objeto*. A continuación, se listan las funciones del primer caso:

Tabla 11: Funciones que no modifican un *array*

Operación	Función	Ejemplo de script	Output (resultado)	Requiere uso de librería
Cantidad de elementos	len()	len(vector)	5	No
Cantidad de elementos	size	vector.size	5	No
Dimensión	ndim	vector.ndim	1	No
Dimensiones de vectores	shape	vector_1.shape	(1,4)	No
Dimensiones de vectores	shape	vector_2.shape	(4,1)	No
Tipo de datos	dtype	vector.dtype	dtype('int64')	No
Mínimo valor	np.max()	np.max(vector)	5	Si
Máximo valor	np.min()	np.min(vector)	1	Si
Sumar los valores	np.sum()	np.sum(vector)	15	Si

Fuente: elaboración propia

Entre las funciones de uso más frecuentes que modifican un *array*, se encuentran:

Tabla 12: Funciones que modifican un *array*

Operación	Función	Ejemplo de script	Output (resultado)	Requiere uso de librería
Añadir un elemento	append()	vector=np.append(vector, 6)	array([1, 2, 3, 4, 5, 6])	Si
Ordenar elementos	sort()	vector0=np.array([4,3,2,1]) vector0.sort()	array([1, 2, 3, 4])	No
Insertar nuevos elementos por posición índice	insert()	vector00=np.array([1,2,4,6]) vector00=np.insert(vector00, [2,3],[3,5], axis=0)	array([1, 2, 3, 4, 5, 6])	Si
Eliminar elementos por posición índice	delete()	vector00=np.delete(vector00, [2,4])	array([1, 2, 4, 6])	Si

Fuente: elaboración propia

Los objetos *array* conformados por números, pueden utilizarse para operar aritméticamente con un escalar u otro *array*. Para ello se utilizan los operadores de la Tabla 2 de la sección 2.2.1 Number. La aplicación de estos operadores es de igual modo con un *array* o vector fila o columna. Por ejemplo, se suma un escalar a un *array*:

```
vector=np.array([1,2,3,4])
vector + 2
Devuelve: array([3, 4, 5, 6])
```

Si ahora se quiere sumar dos *arrays*, se definen:

```
vector=np.array([1,2,3,4])
vector1=np.array([5,6,7])
```

Luego:

```
vector + vector1
```

Devuelve error: **ValueError**: operands could not be broadcast together with shapes (4,) (3,), porque ambos *arrays* deben tener la misma dimensión.

Entonces:

```
vector=np.array([1,2,3,4])
vector2=np.array([5,6,7,8])
vector + vector2
```

Devuelve: array([6, 8, 10, 12])

Si se define un vector fila y un vector columna que coincida la dimensión interna, es decir 1 x 3 y 3 x 1, entonces se obtendrá como resultado una matriz:

```
vector_1=np.array([[1,2,3]])
vector_2=np.array([[1],[2],[3]])
vector_1 + vector_2
```

Devuelve:

```
array([[2, 3, 4],
       [3, 4, 5],
       [4, 5, 6]])
```

Para corroborar lo obtenido, se resuelve una suma escalar:

$$[1,2,3] + \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 2 & 3 & 4 \\ 3 & 4 & 5 \\ 4 & 5 & 6 \end{bmatrix}, \text{ es decir:}$$

$$[1] + \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix} \text{ siendo la primera columna de la matriz}$$

$$[2] + \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 3 \\ 4 \\ 5 \end{bmatrix} \text{ siendo la segunda columna de la matriz}$$

$$[3] + \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix} \text{ siendo la tercera columna de la matriz}$$

De igual modo será con el resto de las operaciones aritméticas, respondiendo a una lógica algebraica.

Ahora bien, si lo que se desea calcular es lo que se denomina “producto escalar” entre vectores, entonces se deberá utilizar la función *dot()*:

```
vector=np.array([1,2,3,4])
vector2=np.array([5,6,7,8])
np.dot(vector,vector2)
```

Se obtiene `np.int64(70)`, lo que se corresponde con:

$$[1,2,3,4] * [5,6,7,8] = (1 * 5) + (2 * 6) + (3 * 7) + (4 * 8) = 5 + 12 + 21 + 32 = 70$$

De igual modo, se puede obtener un producto escalar entre un vector fila y un vector columna:

```
vector_1=np.array([[1,2,3]])
vector_2=np.array([[1],[2],[3]])
np.dot(vector_1,vector_2)
```

Se obtiene `np.int64(14)`, lo que se corresponde con:

$$[1,2,3] * \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} = (1 * 1) + (2 * 2) + (3 * 3) = 14$$

Una operación lógica de utilidad suele ser filtrar elementos de un *array* ya que estos serán como una columna de una tabla de datos. Por ejemplo, del array `[1,2,3,4]` se quiere obtener solo aquellos valores múltiplo de dos, entonces:

```
vector[(vector % 2 == 0)]
```

Donde, con “`% 2 == 0`” se está indicado que el resto de dividir por dos sea cero. Devuelve `array([2, 4])`

También es posible utilizar los operadores lógicos de la Tabla 4 de la sección 2.3.3. Por ejemplo:

```
vector[vector!=2]
Devuelve: array([1, 3, 4])
```

```
vector[vector==2]
Devuelve: array([2])
```

```
vector[vector>2]
Devuelve: array([3, 4])
```

```
vector[vector>=2]
Devuelve: array([2, 3, 4])
```

```
vector[vector<2]
Devuelve: array([1])
```

```
vector[vector<=2]
Devuelve: array([1,2])
```

Ejemplo 2.3.4.1: Cree un programa donde el usuario ingrese los valores entre 5 y 50, cada 10, se cree un array y que devuelva true o false al verificar si todos los valores son múltiplo de 5.

El código por ejecutar es:

```
vector=np.arange(int(input('ingrese el valor inicial: ')),int(input('ingrese el valor final: ')),int(input('ingrese el valor de salto: ')))
verif=vector[(vector % 5 == 0)]>0
```

```
print('Los elementos del vector que es de '+str(vector.ndim)+'D son
múltiplo de 5: '+str(verif.all()))
```

El resultado se verá como sigue:

```
ingrese el valor inicial: 5
ingrese el valor final: 50
ingrese el valor de salto: 10
Los elementos del vector que es de 1D son múltiplo de 5: True
```

Ejemplo 2.3.4.2: Cree un programa donde a partir de la cantidad producida [500, 300, 200] celulares, tablets y computadoras respectivamente; los insumos requeridos para la producción de cada producto por unidad (chip, pantalla, horas de trabajo respectivamente): celulares: [1,1,2] tablets: [1,1,3] computadoras: [2,1,5] y los costos monetarios de producción [45,30,15] devuelva el costo total de producción involucrado.

El código a ejecutar es:

```
P = np.array([500, 300, 200])
V_celular = np.array([1, 1, 2])
V_tablet = np.array([1, 1, 3])
V_computadora = np.array([2, 1, 5])
Costo_prod = np.array([45, 30, 15])
total_recursos = (P[0] * V_celular) + (P[1] * V_tablet) + (P[2] * V_computadora)
costo_total = np.dot(total_recursos, Costo_prod)
print('El costo total de producción es $'+str(costo_total))
```

El resultado es: El costo total de producción es \$127500

Nota: En el siguiente [LINK](#) podrá encontrar un cuaderno ejecutable (Sección 2.3.4 Array) donde se muestra cómo operar con los arreglos a través de Python.

2.3.5 Matrix

Una matriz se define como un vector de dimensión mayor que uno, ya que posee al menos dos filas y dos columnas. De aquí que se trate de un “arreglo rectangular de números” (Chiang y Wainwright, 2006). Su representación es como sigue:

$$A = \begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{n1} & \cdots & a_{nn} \end{bmatrix}$$

Así, la matriz A, posee dimensión m x n. De este modo, es posible comprender que, una matriz en *Python* se trata de un array multidimensional. Las dimensiones vendrán dadas por la cantidad de filas y columna que posee. Los elementos que la conforman deben ser del mismo tipo y serán mutables. Para poder definir y operar

con un *array* multidimensional en *Python*, se deberá utilizar la librería *numpy*. Por ejemplo:

```
import numpy as np
```

Se crea un *array* de dimensión 3 x 2, es decir, tres filas y dos columnas:

```
matrix = np.array([[1, 2],[3, 4],[5, 6]])  
matrix
```

```
Devuelve: array([[1, 2],  
                 [3, 4],  
                 [5, 6]])
```

También es posible crear un *array* multidimensional a partir de *arrays*, listas o tuplas. Por ejemplo:

```
arr1=np.array([1,2])  
arr2=np.array([3,4])  
arr3=np.array([5,6])  
matrix1=np.array([arr1,arr2,arr3])  
matrix1
```

```
Devuelve: array([[1, 2],  
                 [3, 4],  
                 [5, 6]])
```

```
list1=[1,2]  
list2=[3,4]  
list3=[5,6]  
matrix2=np.array([list1,list2,list3])  
matrix2
```

```
Devuelve: array([[1, 2],  
                 [3, 4],  
                 [5, 6]])
```

```
tup1=(1,2)  
tup2=(3,4)  
tup3=(5,6)  
matrix3=np.array([tup1,tup2,tup3])  
matrix3
```

```
Devuelve: array([[1, 2],  
                 [3, 4],  
                 [5, 6]])
```

También es posible definir una matriz a partir de un único array, utilizando la función *reshape()*. Por ejemplo:

```
arr = np.array([1, 2, 3, 4, 5, 6])  
matrix4 = arr.reshape(3, 2)
```

```
matrix4
```

```
Devuelve: array([[1, 2],
                 [3, 4],
                 [5, 6]])
```

Ahora bien, la función *reshape()* también puede utilizarse para convertir una matriz en un array unidimensional. Por ejemplo:

```
vector=matrix.reshape(-1)
vector
```

```
Devuelve array([1, 2, 3, 4, 5, 6])
```

Al igual que las listas y los arreglos, un *array* multidimensional tiene un índice implícito en *Python*. Pero ahora serán dos índices, uno para las filas y otro para las columnas. Esto quiere decir que, para obtener los elementos del objeto de manera individualizada, se debe citar la posición de cada índice.

	Índice implícito columnas	
Índice implícito filas	0	1
0	1	2
1	3	4
2	5	6

Por ejemplo, se obtiene el primer elemento del objeto `matrix` creado inicialmente:

```
matriz[0,0]
```

donde el cero de la izquierda corresponde a la posición en el índice de la fila y el cero de la derecha corresponde a la posición en el índice de la columna. Devuelve: `np.int64(1)`

Como se mencionó, un *array* multidimensional es mutable, razón por la cual es posible modificarlo o modificar sus elementos. En este último caso, se deberá indicar la posición del elemento que se quiere modificar y asignar el valor correspondiente. Por ejemplo, para modificar el último elemento de la matriz:

```
matriz[2,1]=10
```

```
Ahora se observará que devuelve: array([[1, 2],
                                         [3, 4],
                                         [5, 10]])
```

También es posible reemplazar de a varios elementos a la vez. Por ejemplo, se modifica la última fila:

```
matrix[2,:]=[5,6]
```

```
Ahora se observará que devuelve: array([[1, 2],
                                         [3, 4],
                                         [5, 6]])
```

También se puede modificar una columna. Por ejemplo, se modifica la última columna:

```
matrix[0:3,1:2]=[[10],[11],[12]]
```

Ahora se observará que devuelve: `array([[1, 10],
[3, 11],
[5, 12]])`

Para agregar un nuevo elemento fila o columna, se deberán utilizar las funciones `hstack()` y `vstack()` respectivamente dependientes de la librería `numpy`. Por ejemplo, se agrega una nueva columna a la matriz original:

```
matrix0 =np.hstack((matrix, [[10],[11],[12]]))
```

Devuelve: `array([[1, 2, 10],
[3, 4, 11],
[5, 6, 12]])`

También se puede agregar una fila del siguiente modo:

```
matrix0 =np.vstack((matrix, [[10,11]]))
```

Devuelve: `array([[1, 2],
[3, 4],
[5, 6],
[10, 11]])`

Las funciones más frecuentes de utilizar con un array multidimensional que no modifican el objeto son las mismas ya presentadas en la Tabla 11 de la sección anterior. Por otro lado, se encuentran las funciones particulares que permiten obtener el determinante, una matriz inversa, entre otros. Entre estas, las de uso más frecuentes son:

Tabla 13: Funciones particulares

Operación	Función	Ejemplo de script	Output (resultado)	Requiere uso de librería
Matriz traspuesta	T	<code>matrix.T</code>	<code>array([[1, 3, 5], [2, 4, 6]])</code>	No
Determinante	<code>linalg.det()</code>	<code>matrix_cuadrada=np.array([[1,2],[3,4]])</code> <code>np.linalg.det(matrix_cuadrada)</code>	<code>np.float64(-2)</code>	Si
Diagonal	<code>diagonal()</code>	<code>matrix.diagonal()</code>	<code>array([1, 4])</code>	No
Traza	<code>trace()</code>	<code>matrix.trace()</code>	<code>array([5])</code>	Si
Matriz invertida	<code>linalg.inv()</code>	<code>np.linalg.inv(matrix_cuadrada)</code>	<code>array([[-2. , 1.], [1.5, -0.5]])</code>	Si
Valores propios	<code>linalg.eig()</code>	<code>eigenvalues=np.linalg.eig(matrix_cuadrada)[0]</code>	<code>array([-0.37228132, 5.37228132])</code>	Si

Vectores propios	<code>linalg.eig()</code>	<code>eigenvectors=</code> <code>np.linalg.eig(matrix_cuadrada)[1]</code>	<code>array([[-0.82456484, -0.41597356], [0.56576746, -0.90937671]])</code>	Si
------------------	---------------------------	--	---	----

Fuente: elaboración propia

Los objetos *array* multidimensionales conformados por números, pueden utilizarse para operar aritméticamente con un escalar u otro *array*. Para ello se utilizan los operadores aritméticos. La aplicación de estos operadores es de igual que para un *array* unidimensional. Por ejemplo, se suma un escalar a una matriz:

```
matrix_cuadrada=np.array([[1,2],[3,4]])
matrix_cuadrada + 3
Devuelve: array([[4, 5],
                 [6, 7]])
```

Si ahora se quiere sumar dos matrices, las mismas deben tener la misma dimensión. Entonces:

```
matrix_cuadrada_2=np.array([[5,6],[7,8]])
matrix_cuadrada + matrix_cuadrada_2
Devuelve: array([[ 6,  8],
                 [10, 12]])
```

Para corroborar lo obtenido, se resuelve una suma escalar:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} + \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 1+5 & 2+6 \\ 3+7 & 4+8 \end{bmatrix} = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}$$

De igual modo será con el resto de las operaciones aritméticas, respondiendo a una lógica algebraica.

Ahora bien, si lo que se desea calcular es lo que se denomina “producto escalar” entre matrices, entonces se deberá utilizar la función *dot()*:

```
np.dot(matrix_cuadrada,matrix_cuadrada_2)
```

```
Se obtiene: array([[ 19,  22],
                  [43, 50]])
```

lo que se corresponde con:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} * \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} (1*5) + (2*7) & (1*6) + (2*8) \\ (3*5) + (4*7) & (3*6) + (4*8) \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

Una operación lógica de utilidad, suele ser filtrar elementos de un *array* multidimensional ya que estos serán tabla de datos. Por ejemplo, de *matrix_cuadrada* se quiere obtener solo aquellos valores múltiplo de dos, entonces:

```
matrix_cuadrada[(matrix_cuadrada % 2 == 0)]
```

Donde, con “% 2 == 0” se está indicado que el resto de dividir por dos sea cero. Devuelve: `array([2, 4])`

También es posible utilizar los operadores lógicos de la Tabla 4 de la sección 2.3.3. del mismo modo que con un array unidimensional. Por ejemplo:

```
matrix_cuadrada [matrix_cuadrada!=2]
```

Devuelve: array([1, 3, 4])

Ejemplo 2.3.4.1: Construya una matriz con los siguientes vectores fila: (1,2,3,4), (5,6,7,8), (9,10,11,None), (13,None,15,None). Realice las siguientes operaciones en celdas individuales:

- Verifique las dimensiones
- Obtenga el elemento que se encuentra en la fila 2 y columna 3
- Reemplace el primer valor None por 12, el segundo None por 14 y el tercero por 16.
- Verifique si los elementos son negativos
- Calcule la suma de las columnas y asigne el resultado a un vector denominado vector1.

El código por ejecutar es:

```
matrix=np.array([[1,2,3,4],
                 [5,6,7,8],
                 [9,10,11,None],
                 [13,None,15,None]])

matrix.shape
matrix[1,2]
matrix[2,3]=12
matrix[3,1]=14
matrix[3,3]=16
matrix<0
vector1=np.sum(matrix, axis=0)
```

El resultado en cada instancia se verá como sigue:

```
array([[1, 2, 3, 4],
       [5, 6, 7, 8],
       [9, 10, 11, None],
       [13, None, 15, None]], dtype=object)

(4, 4)
7
array([[1, 2, 3, 4],
       [5, 6, 7, 8],
       [9, 10, 11, 12],
       [13, 14, 15, 16]], dtype=object)
array([[False, False, False, False],
       [False, False, False, False],
       [False, False, False, False],
       [False, False, False, False]])
array([28, 32, 36, 40], dtype=object)
```

Ejemplo 2.3.4.2: Un inversor posee \$12.000 para invertir en tres fondos diferentes. Los fondos presentan los siguientes rendimientos: fondo de renta fija paga un 3 % de interés anual; fondo de renta variable paga un 4 % anual; fondo de criptoactivos

que pagan un 7 % anual. Sabiendo que invertirá \$4.000 más en criptoactivos que en fondos variables y que el interés total a obtener en un año es \$670. ¿Qué monto invertirá en cada tipo de fondo?

Se define:

X= fondo de renta fija; Y=fondo de renta variable; Z=fondo de criptoactivos

A partir de los datos dados, se plantea el siguiente sistema de ecuaciones:

$$X + Y + Z = 12000$$

$$Z = 4000 + Y \Rightarrow -Y + Z = 4000$$

$$0,03X + 0,04Y + 0,07Z = 670$$

$$\begin{cases} X + Y + Z = 12000 \\ -Y + Z = 4000 \\ 0,03X + 0,04Y + 0,07Z = 670 \end{cases}$$

La solución del sistema vendrá dada por:

$$x = A^{-1} * d \text{ siendo } A = \begin{bmatrix} 1 & 1 & 1 \\ 0 & -1 & 1 \\ 0,03 & 0,04 & 0,07 \end{bmatrix} \text{ y } d = \begin{bmatrix} 12000 \\ 4000 \\ 670 \end{bmatrix}$$

El código a ejecutar es:

```
A=np.array([[1,1,1],
            [0,-1,1],
            [0.03,0.04,0.07]])

invA=np.linalg.inv(A)
d=np.array([12000,4000,670])
x=np.dot(invA,d)
print('La distribución de los montos a invertir son:
$'+str(round(x[0]))+' en fondos de renta fija,
$'+str(round(x[1]))+' en fondos de renta variable,
$'+str(round(x[2]))+' en fondos de criptoactivos')
```

El resultado es:

La distribución de los montos a invertir son: \$2000 en fondos de renta fija, \$3000 en fondos de renta variable, \$7000 en fondos de criptoactivos

Para verificar el resultado obtenido, se deberá recordar que:

$A^{-1} = \frac{1}{|A|} adjA$ donde $|A|$ es el determinante de la matriz A y $adjA$ es la matriz de cofactores traspuesta.

Entonces:

$$adjA = \begin{bmatrix} 1 * \begin{vmatrix} -1 & 1 \\ 0,04 & 0,07 \end{vmatrix} & -1 * \begin{vmatrix} 0 & 1 \\ 0,03 & 0,07 \end{vmatrix} & 1 * \begin{vmatrix} 0 & -1 \\ 0,03 & 0,04 \end{vmatrix} \\ -1 * \begin{vmatrix} 1 & 1 \\ 0,04 & 0,07 \end{vmatrix} & 1 * \begin{vmatrix} 1 & 1 \\ 0,03 & 0,07 \end{vmatrix} & -1 * \begin{vmatrix} 1 & 1 \\ 0,03 & 0,04 \end{vmatrix} \\ 1 * \begin{vmatrix} 1 & 1 \\ -1 & 1 \end{vmatrix} & -1 * \begin{vmatrix} 1 & 1 \\ 0 & 1 \end{vmatrix} & 1 * \begin{vmatrix} 1 & 1 \\ 0 & -1 \end{vmatrix} \end{bmatrix}^T =$$

$$\begin{bmatrix} -0,11 & 0,03 & 0,03 \\ -0,03 & 0,04 & -0,01 \\ 2 & -1 & -1 \end{bmatrix}^T = \begin{bmatrix} -0,11 & -0,03 & 2 \\ 0,03 & 0,04 & -1 \\ 0,03 & -0,01 & -1 \end{bmatrix}$$

$$|A| = 1 * \begin{vmatrix} -1 & 1 \\ 0,04 & 0,07 \end{vmatrix} - 1 * \begin{vmatrix} 0 & 1 \\ 0,03 & 0,07 \end{vmatrix} + 1 * \begin{vmatrix} 0 & -1 \\ 0,03 & 0,04 \end{vmatrix} = -0,11 + 0,03 + 0,03 = -0,05$$

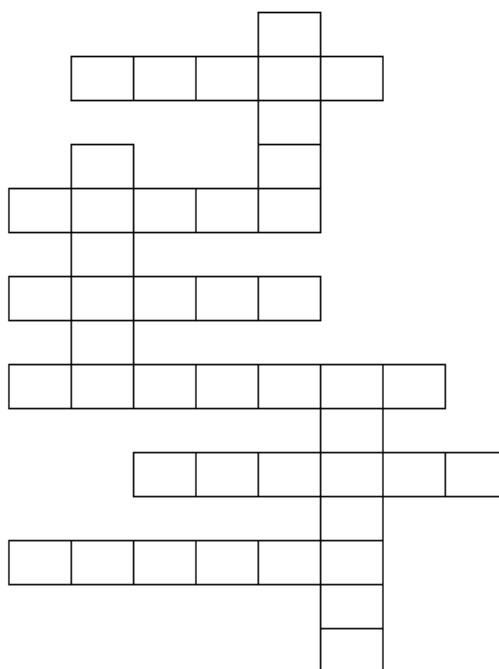
$$A^{-1} = \frac{1}{-0,05} \begin{bmatrix} -0,11 & -0,03 & 2 \\ 0,03 & 0,04 & -1 \\ 0,03 & -0,01 & -1 \end{bmatrix} = \begin{bmatrix} 2,2 & 0,6 & -40 \\ -0,6 & -0,8 & 20 \\ -0,6 & 0,2 & 20 \end{bmatrix}$$

$$x = \begin{bmatrix} 2,2 & 0,6 & -40 \\ -0,6 & -0,8 & 20 \\ -0,6 & 0,2 & 20 \end{bmatrix} * \begin{bmatrix} 12000 \\ 4000 \\ 670 \end{bmatrix} = \begin{bmatrix} 2000 \\ 3000 \\ 7000 \end{bmatrix}$$

Nota: En el siguiente [LINK](#) podrá encontrar un cuaderno ejecutable (Sección 2.3.5 Matrix) donde se muestra cómo operar con las matrices a través de Python.

2.4. Ejercicios propuestos

Ejercicio 2.4.1: complete el siguiente crucigrama a partir de las definiciones dadas



Definiciones horizontales

- Secuencia de elementos no mutables
- Valor True o Falso
- Array de dos dimensiones o más
- Cadena de caracteres
- Valor None
- Momento preciso en un calendario

Definiciones verticales

- Secuencia de elementos mutable que admite diferente tipo de dato.
- Dígito o cifra
- Secuencia de elementos mutable que no admite diferente tipo de dato.

Ejercicio 2.4.2: cree un programa que, a partir de que el usuario ingrese su nombre y apellido, su nacionalidad y su edad, conforme una única lista. El nombre, el apellido y la nacionalidad conviértalos a mayúscula. Con la edad calcule el año de nacimiento. Finalmente, que el programa devuelva “El ciudadano NOMBRE y APELLIDO de nacionalidad NACIONALIDAD nació en el año AÑO”

Ejercicio 2.4.3: cree un programa donde defina una lista con los siguientes productos: A, B, C, D, E, F; y defina otra lista con los precios 10, 25, 38, 50, 62, 75. Ahora almacene el nombre de cada producto por un lado y el valor de cada cantidad comprada por otro, a partir de que los ingrese el usuario para tres de los productos. Finalmente, que el programa devuelva “El cliente compró X unidades de X a \$X, Y unidades de Y a \$Y, Z unidades de Z a \$Z y en total gastó \$MONTO”.

Ejercicio 2.4.4: Los clientes tenedores de crédito de un banco se clasifican en las siguientes categorías: “A” (Tradicionales) siendo los que utilizan los canales tradicionales del banco, “B” (Tecnológicos) siendo los que utilizan los canales virtuales del banco, mientras que la categoría “C” (Ambos) contiene a los que utilizan un mix de ambos canales. A su vez, si luego de un proceso de reclamo judicial no responden, se los clasifica en la categoría “D” (Default) y se supone que no se recupera nada de lo adeudado. Mientras que si cancelan la deuda se los clasifica en la categoría “P” (pagado).

El banco realiza un análisis mensual de la clasificación de los tenedores de crédito. En base a un análisis histórico, se estima que el 30% pertenecen “A” seguirán en la misma categoría en el mes siguiente, mientras que un 20% pasarán a la categoría “B”, 10% a la categoría “C” y el resto pagará completamente el saldo deudor.

Por otra parte, se estima que un 10% de los tenedores de crédito en la categoría “B” caerán en default en el mes siguiente, mientras que un 20% responderá a los reclamos cancelando completamente su deuda. Asimismo, se estima que un 30% continuará en la misma categoría, mientras que el 30% pasará a la categoría “A” y el 10% restante pasará a “C”. En el caso de los de categoría “C”, se estima que un 40% seguirá en la misma categoría el mes siguiente, que un 50% finalmente cancelará la deuda debido a una renegociación y que solo un 10 % caerá en Default. Suponga que la evolución de los saldos deudores se puede analizar mediante una cadena de Markov. A partir ello, resuelva: Si el banco actualmente tiene créditos por cobrar por \$ 150.000 clasificados como “A”, \$ 60.000 clasificados como “B” y

\$25.000 clasificados como “C” (todos los montos en millones de pesos), ¿cuánto esperarías cobrar el banco y cuánto se consideraría en Default?

Nota: para resolver este ejercicio, se sugiere revisar el tema Cadenas de Markov Discretas¹⁰ y resolver por el método matricial. Luego, ver la librería PyDTMC¹¹ de *Python*.

3. Programación funcional con Python

En los capítulos anteriores, se utilizaron **funciones** pertenecientes a una librería y otras que no. De aquí que, poder contar con funciones ya programadas facilita la operabilidad para la implementación de códigos. Esto porque, en reiteradas ocasiones, se requiere realizar una o más veces una misma operación. Particularmente, cuando se trabaja con datos cotidianamente, se suelen realizar tareas que son repetitivas. En este sentido, poder armar estructuras de código que sean reutilizables, suele ser una ventaja para ganar tiempo de acción. De esta manera, poder conocer cómo crear funciones en *Python*, que se ajusten a una necesidad particular, cobrará relevancia.

Al mismo tiempo, las **estructuras funcionales** suelen contribuir a poder realizar una lectura más clara de un bloque de código (McKinney, 2018), principalmente cuando involucra multiplicidad de instrucciones. Ahora bien, para poder instruir a un lenguaje con múltiples acciones detalladas, se requiere utilizar estructuras lógicas. Estas permitirán especificar el paso a paso de manera estructurada hasta poder obtener un resultado. En definitiva, permitirá llevar una estructura abstracta de pasos, como las presentadas en el capítulo 1 mediante diagramas de flujos, a un programa ejecutable.

Entre los pasos necesarios a detallar, se deberá especificar la evaluación de condiciones ya sea para una opción de si o no, o para recorrer cada elemento de un objeto y sobre estos evaluar. Por esta razón, los **operadores** IF, FOR y WHILE, serán las instrucciones básicas necesarias para lograrlo. Una vez que se diseña una lógica programática base de este tipo, luego podrá ser incorporado en un bloque estructurado de código que permita generalizar el proceso, es decir, convertirlo en funcional para ser reutilizado.

3.1. Condicional IF

Hasta ahora, se han ejecutado instrucciones de manera lineal o directa. Si se requería sumar, simplemente se sumaba; o si se requería realizar una indexación (acceder a elementos a través de índice), se especificaba una posición y se obtenía lo buscado. Esta forma de accionar no tiene en consideración distintas posibilidades que pueden presentarse o se requieran evaluar. Así, por ejemplo, si solo se requiere sumar los números pares en una secuencia de números dados, será necesario poder especificarle tal condición. También, surgirá la necesidad de

¹⁰ Acerca de Cadenas de Markov ver capítulo 4 en Sheldon Ross (2014) <https://essaywritinglebanon.com/wp-content/uploads/2021/05/introduction-to-prob-models-11th-edition.pdf>

¹¹ Acerca PyDTMC <https://pydtmc.readthedocs.io/> ; <https://github.com/TommasoBelluzzo/PyDTMC>

crear nuevas variables cuando se trabaje con tablas de datos que, en ocasiones, dependerá de condiciones sobre otras variables existentes.

Para poder establecer condiciones en una lógica de pasos, se utiliza el condicional IF. Este permite construir lógicas operativas del tipo “SI pasa tal cosa, entonces hacer tal operación”. A partir de aquí, es que su estructura consistirá en, como mínimo, tres partes (Guttaj, 2017; McKinney, 2018): una expresión que se evalúa como Verdadero o Falso; una instrucción mediante código que se ejecuta si la expresión resulta ser verdadera; una instrucción que se ejecuta si la expresión resulta ser falsa. Llevado a bloque de código, se escribirá como sigue:

```
if condición1:
    realizar acción1
else:
    realizar acción3
```

El bloque anterior se leerá del siguiente modo:

SI (IF) condición 1 es verdadera, entonces (:) se realizará la acción 1

SINO (ELSE), (:) se realizará la acción 3

Para construir las condiciones a ser evaluadas, se deberá utilizar los operadores vistos en la Tabla 4 de la sección 2.2.3

Al escribir un bloque de código como el anterior, se agrega **indentación**. De hecho, cuando se escriba el código, luego de los dos puntos al apretar “Enter”, la indentación se generará automáticamente. De aquí que se trata del espacio generado en el siguiente nivel a los dos puntos, es decir, en la línea donde se especifica la acción a ejecutar. La indentación en *Python* adquiere significado semántico (Guttaj, 2017). Esto es, brinda garantía de que la estructura visual del código represente la lógica de pasos de un modo claro.

También, será posible evaluar múltiples instancias de condiciones verdaderas a través de utilizar el operador *elif*. Por ejemplo, para una estructura que evalúe dos condiciones verdaderas, el bloque de código a escribir es:

```
if condición1:
    realizar acción1
elif condición2:
    realizar acción2
else:
    realizar acción3
```

Se podrán agregar tantos niveles con *elif* como condiciones verdaderas a evaluar. Además, en las condiciones que se especifiquen, podrá establecerse condiciones compuestas, es decir, evaluar más de una cosa como verdadera. Para ello se utilizará el operador “and” u “or” o una combinación de ambos. Por ejemplo:

<pre>if x<y and x<z: realizar acción1 elif y<z:</pre>	<pre>if x<y or x<z: realizar acción1 elif y<z:</pre>	<pre>if x<y or (x<z and t>z): realizar acción1 elif y<z and t<z:</pre>
--	---	---

realizar acción2	realizar acción2	realizar acción2
else:	else:	else:
realizar acción3	realizar acción3	realizar acción3

Otra posibilidad es crear condicionales anidados. Por ejemplo:

```
if x<y and x<z:
    if y<z:
        realizar acción1
    else:
        pass
elif y<z:
    realizar acción2
else:
    realizar acción3
```

En el bloque anterior se utilizó el operador *pass*. Este permite indicar que no se haga nada y se continuará ejecutando el código restante.

Ejemplo 3.1.1: Cree un programa a partir del diagrama de flujo del ejemplo 1 del capítulo 1 que devuelva si la edad ingresada por el usuario es un número es par o no.

El código a ejecutar es:

```
x = input('Ingrese su edad: ')
if x.isdigit():
    x=int(x)
    if x % 2 == 0 :
        print('Su edad es un número par')
    else:
        print('Su edad es un número impar')
else:
    print('Debe ingresar un número entero')
```

Si por ejemplo se ingresa 21, devuelve: Su edad es un número impar

Si por ejemplo se ingresa 21.5, devuelve: Debe ingresar un número entero

Si por ejemplo se ingresa 22, devuelve: Su edad es un número par

Notar que se utilizó la función *isdigit()* para saber si el contenido ingresado es un elemento conformado por dígitos (números), dado que *input()* devuelve un *string*.

Ejemplo 3.1.2: Cree un programa donde el usuario ingrese un número y devuelva si es número primo o no.

Recordar:

- Si es un número negativo, 0 o 1, no es primo
- Si es 2 es primo
- Si es mayor que 2 y divisible por 3, 5 o 7, es primo
- Si es mayor que 2 y múltiplo de 2, 3, 5 o 7, no es primo
- En cualquier otro caso, es primo

- Solo los números enteros positivos pueden ser primo.

El código a ejecutar es:

```
num=input('Introduzca un número: ')
if num.isdigit():
    x=int(num)
    if x < 0 or x==0 or x==1:
        print('El número '+str(x)+' no es primo')
    elif x==2:
        print('El número '+str(x)+' es primo')
    elif x>2 and x%x==0 and (x/3==1 or x/5==1 or x/7==1):
        print('El número '+str(x)+' es primo')
    elif x>2 and x%x==0 and (x % 2==0 or x % 3==0 or x % 5==0 or x % 7==0):
        print('El número '+str(x)+' no es primo')
    else:
        print('El número '+str(x)+' es primo')
else:
    print('Debe ingresar un número entero positivo')
```

Si por ejemplo se ingresa 0.5, devuelve: Debe ingresar un número entero positivo

Si por ejemplo se ingresa -1, devuelve: Debe ingresar un número entero positivo

Si por ejemplo se ingresa 0, devuelve: El número 0 no es primo

Si por ejemplo se ingresa 2, devuelve: El número 2 es primo

Si por ejemplo se ingresa 3, devuelve: El número 3 es primo

Si por ejemplo se ingresa 4, devuelve: El número 4 no es primo

Si por ejemplo se ingresa 7, devuelve: El número 7 es primo

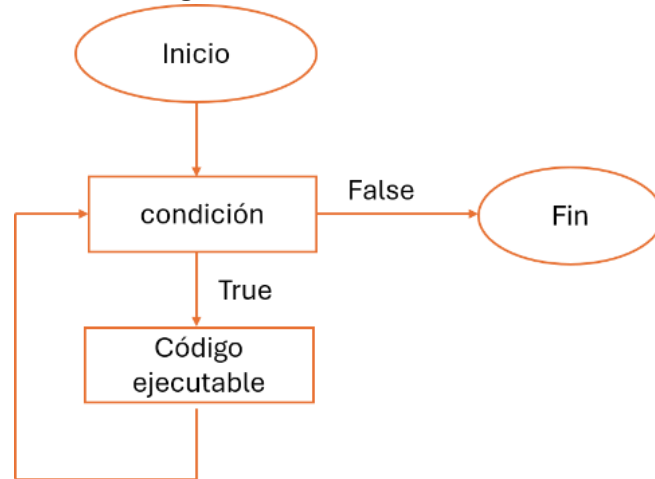
Nota: En el siguiente [LINK](#) podrá encontrar un cuaderno ejecutable (Sección 3.1 Condicional IF) donde se muestra cómo operar a través de Python un condicional.

3.2. Bucle WHILE

Una primera forma de accionar iterativamente es a través del uso del operador WHILE (“mientras”). Este facilita repetir una acción varias veces mientras se cumpla alguna condición hasta que sea falsa (Guttaj, 2017; McKinney, 2018). Esto es, brinda la posibilidad de crear un proceso evaluativo con validación sobre elementos dados hasta que se cumpla con el objetivo establecido y finaliza el proceso.

La lógica operativa de un bucle WHILE, requiere de dos cosas claves: la primera, es la inicialización de una variable en cero; la segunda, es la actualización de dicha variable a medida que el proceso iterativo transcurre. Esta actuará como un contador que se incrementa en cada interacción. Su estructura viene dada por:

Figura 4: Estructura WHILE



Fuente: elaboración propia

En términos de código, será

```
contador=0
while condición1:
    bloque de código ejecutable
    contador+=1
else:
    print('Trabajo finalizado')
```

Por ejemplo, si se quiere imprimir uno por uno los números del 1 al 10 inclusive, entonces:

```
contador = 0
x=1
while contador < 10:
    print(x)
    x+=1
    contador+=1
else:
    print('Trabajo finalizado')
```

Devuelve:

```
1
2
3
4
5
6
7
8
9
10
Trabajo finalizado
```

El accionar del código anterior es como sigue: la primera iteración inicia con el contador en 0. Dado que cumple con ser menor que 10, entonces ejecuta lo

indicado y devuelve 1 siendo el primer valor asignado a x. Además, se actualiza el contador al sumarle uno, es decir, pasa a valer 1. En la segunda iteración, el contador vale 1 siendo menor a 10 por lo que devuelve *True*, entonces ejecuta el *print* devolviendo el valor 2 ya que ahora el x vale 2 debido a la actualización anterior. Así sucesivamente hasta que el contador toma el valor de 11. En este momento no se cumple la condición, entonces ejecuta la parte de *else* y finaliza. Si se representa esta lógica en un cuadro, entonces:

Iteración	contador	x	condición	evaluación	Output	contador	x
1	0	1	0<10	True	1	0+1=1	1+1=2
2	1	2	1<10	True	2	1+1=2	2+1=3
...	...	...	...	...	...	...	...
10	9	10	9<10	True	10	9+1=10	10+1=11
11	10	11	10<10	False	Trabajo finalizado		

También es posible construir un iterador WHILE que incluya un condicional IF. Al ejemplo anterior, ahora se agrega cumplir con la siguiente condición: cuando x sea igual a 5, entonces que el proceso finalice. Entonces el código vendrá dado por:

```
contador = 0
x=1
while contador < 10:
    print(x)
    x+=1
    contador+=1
    if x==3:
        print('Objetivo alcanzado')
        break
    else:
        continue
else:
    print('Trabajo finalizado')
```

Lo que devuelve:

```
1
2
Objetivo alcanzado, x=3
```

Ejemplo 3.2.1: Sabiendo que una contraseña es "123456", cree un programa que pregunte al usuario por su contraseña y finalice cuando haya introducido la contraseña correcta.

El código a ejecutar es:

```
key = "123456"
password = ""
while password != key:
    password = input("Introduce la contraseña: ")
else:
```

```
print("Contraseña correcta")
```

Si por ejemplo se ingresa “hola”, vuelve a solicitar ingreso de contraseña

Introduce la contraseña: hola

Si por ejemplo se ingresa “123456”, devuelve

Introduce la contraseña: 123456

Contraseña correcta

Notar que no se utilizó un contador, sino que directamente se evaluó una condición como verdadera para obtener una iteración.

Ejemplo 3.2.2: Cree un programa donde, a partir de que el usuario ingrese un monto a invertir y el número de años, calcule el capital más interés obtenido cada año de manera acumulativa sabiendo que se paga un 10% anual. Además, deberá tener en cuenta que el capital acumulado no puede superar en más del 100% al capital inicial. Si esto sucede, debe finalizar el proceso.

El código a ejecutar es:

```
montoinicial = float(input("Cantidad a invertir: "))
años = int(input("Años: "))
interes = 10
i=0
montoacum=montoinicial
while montoacum <= montoinicial*2:
    montoacum = montoacum + (montoacum * interes / 100)
    if montoacum < montoinicial*2:
        print("Capital año " + str(i+1) + " años: " + str(round(montoacum, 2)))
        i+=1
    else:
        print('Se alcanzó el límite de capitalización permitido')
```

Si por ejemplo se ingresa 1000 de capital y 10 años de plazo, devuelve:

Cantidad a invertir: 1000

Años: 10

Capital año 1 años: 1100.0

Capital año 2 años: 1210.0

Capital año 3 años: 1331.0

Capital año 4 años: 1464.1

Capital año 5 años: 1610.51

Capital año 6 años: 1771.56

Capital año 7 años: 1948.72

Se alcanzó el límite de capitalización

Nota: En el siguiente [LINK](#) podrá encontrar un cuaderno ejecutable (Sección 3.2 Bucle WHILE) donde se muestra cómo operar a través de Python un interador WHILE.

3.3. Bucle FOR

Una forma de iterar sobre una colección de elementos es a través del bucle FOR. A diferencia del bucle WHILE, este no requiere que una condición se cumpla, sino que permite recorrer cada elemento contenido en un objeto y operar sobre él. Esto es, se busca repetir una instrucción u operación sobre cada elemento. De esta manera, el bucle FOR permitirá simplificar programas que requieren de un proceso iterativo (Guttaj, 2017; McKinney, 2018).

Al realizar una operación sobre cada elemento, el resultado obtenido deberá ser almacenado en un objeto. De este modo, luego podrá manipularse dicho objeto, es decir, simplemente observarlo o realizar otra operación agregativa sobre este. Por ejemplo, si se tiene una lista de números, simplemente se puede requerir multiplicar por dos cada elemento e imprimir el resultado; o, una vez que se multiplican los elementos por dos, finalmente obtener la suma de todos los resultados. También, el objetivo puede ser buscar un valor en particular dentro de una secuencia de elementos y finalizar el proceso ante el resultado exitoso.

La estructura del código ejecutable será

```
for i in objeto_iterable:
    bloque de código ejecutable
```

donde se puede observar que se requiere agregar indentación para la instrucción.

Por ejemplo, si tan solo se quiere obtener los elementos de una lista:

```
list1 = [1,2,3,4,5,6,7,8,9,10]
for i in list1:
    print(i)
```

Devuelve:

```
1
2
3
4
5
6
7
8
9
10
```

Algo importante a recordar, es que la lista tiene implícito un índice por columnas. Esto quiere decir, que el FOR recorre los elementos del objeto a través del índice columna.

Por ejemplo, si ahora se agrega una operación sobre cada elemento:

```
list1 = [1,2,3,4,5,6,7,8,9,10]
for i in list1:
    print(i*2)
```

Devuelve:

```
2
4
```


6
8
10
12
14
16
18
20

Si se representa esta lógica en un cuadro, entonces:

Nº Iteración	i	Output
1	1	1*2=2
2	2	2*2=4
...	...	...
10	10	10*2=20

Ahora bien, si por ejemplo se quiere multiplicar cada elemento por dos y finalmente obtener la suma de todos los resultados:

```
list1 = [1,2,3,4,5,6,7,8,9,10]
list2=[]
for i in list1:
    lista2.append(i*2)
sum(lista2)
```

Devuelve: 110

Notar que, la operación final de sumar se realiza por fuera del bloque FOR. Esto es, la operación agregativa final, se realiza por fuera de la lógica iterativa, razón por la cual el resultado de la iteración debe ser almacenada en un objeto. A pesar de ello, la característica simplificadora del FOR, permite realizarse lo anterior de un modo más simple, si crea una variable inicializada en 0:

```
list_sum = 0
for i in list1:
    list_sum = list_sum + (i*2)
list_sum
```

Devuelve: 110. Por esta razón, el bucle FOR, es un facilitador para simplificar programas. En este caso la lógica operativa es:

list_sum	Nº Iteración	i	Operación	list_sum
0	1	1	0+(1*2)	2
2	2	2	2+(2*2)	6
6	3	3	6+(3*2)	12
12	4	4	12+(4*2)	20
20	5	5	20+(5*2)	30
30	6	6	30+(6*2)	42
42	7	7	42+(7*2)	56
56	8	8	56+(8*2)	72
72	9	9	72+(9*2)	90
90	10	10	90+(10*2)	110

También, es posible combinar un iterador FOR con un condicional IF de forma tal de instruir al programa que actúe solo cuando se cumple una condición. Por ejemplo, suponga que de la `list1` solo se quiere obtener aquellos elementos que son pares. Entonces:

```
pares=[]
for i in list1:
    if i % 2 == 0:
        pares.append(i)
    else:
        pass
pares
```

Devuelve: [2, 4, 6, 8, 10]

En este caso la lógica será:

pares	Nº Iteración	i	Condición	Evaluación	pares
[]	1	1	1/2 -> resto ≠ 0	False	[]
[]	2	2	2/2 -> resto = 0	True	[2]
[2]	3	3	3/2 -> resto ≠ 0	False	[2]
[2]	4	4	4/2 -> resto = 0	True	[2,4]
[2,4]	5	5	5/2 -> resto ≠ 0	False	[2,4]
[2,4]	6	6	6/2 -> resto = 0	True	[2,4,6]
[2,4,6]	7	7	7/2 -> resto ≠ 0	False	[2,4,6]
[2,4,6]	8	8	8/2 -> resto = 0	True	[2,4,6,8]
[2,4,6,8]	9	9	9/2 -> resto ≠ 0	False	[2,4,6,8]
[2,4,6,8]	10	10	10/2 -> resto = 0	True	[2,4,6,8,10]

Por defecto, con el operador FOR, *Python* recorrerá el objeto a través del índice columna. Si el elemento es unidimensional, entonces el `i` del FOR devuelve cada elemento contenido, como fue el caso de todo lo realizado hasta el momento. Ahora bien, si el objeto es multidimensional, como en el caso de una matriz, se deberá tener en cuenta que el `i` en el FOR solo le devolverá el vector completo que es la columna. Para acceder a los elementos contenidos en cada columna, se requerirá de una doble indexación, utilizando la función `enumerate()`. Esta permite obtener la posición del índice correspondiente al objeto. Por ejemplo, si crea una matriz con vectores filas:

```
import numpy as np
matrix=np.array([[1,2,3,4], [5,6,7,8], [9,10,11,None],
[13,None,15,None]])
for i, v in enumerate(matrix):
    print(i, v)
```

devuelve:

```
0 [1 2 3 4]
1 [5 6 7 8]
2 [9 10 11 None]
3 [13 None 15 None]
```

Una vez que se logra obtener la posición del índice para objeto, entonces se requerirá otro FOR para acceder a los elementos de cada vector y obtener su posición:

```
for i, v in enumerate(matrix):
    for j, c in enumerate(v):
        print(i, v, j, c)
```

devuelve:

```
0 [1 2 3 4] 0 1
0 [1 2 3 4] 1 2
0 [1 2 3 4] 2 3
0 [1 2 3 4] 3 4
1 [5 6 7 8] 0 5
1 [5 6 7 8] 1 6
1 [5 6 7 8] 2 7
1 [5 6 7 8] 3 8
2 [9 10 11 None] 0 9
2 [9 10 11 None] 1 10
2 [9 10 11 None] 2 11
2 [9 10 11 None] 3 None
3 [13 None 15 None] 0 13
3 [13 None 15 None] 1 None
3 [13 None 15 None] 2 15
3 [13 None 15 None] 3 None
```

donde la primera columna de números repetidos es el índice de las filas donde se posiciona cada vector; la tercera columna de números es el índice de las columnas para cada elemento contenidos en cada vector. A partir de ello, si se requiere reemplazar los valores *None* por cero, entonces:

```
for i, v in enumerate(matrix):
    for j, c in enumerate(v):
        if c is None:
            matrix[i][j] = 0
        else:
            pass
```

matrix

devuelve:

```
array([[1, 2, 3, 4],
       [5, 6, 7, 8],
       [9, 10, 11, 0],
       [13, 0, 15, 0]], dtype=object)
```

Ejemplo 3.3.1: Construya una matriz con los siguientes vectores fila: (1,2,3,4), (5,6,7,8), (9,10,11, None), (13, None, 15, None). Reemplace el primer valor None por 12, el segundo None por 14 y el tercero por 16.

Nota: observar que se está resolviendo el punto d) del Ejemplo 2.3.4.1 de manera automatizada.

El código a ejecutar es:

```
import numpy as np
matrix=np.array([[1,2,3,4], [5,6,7,8], [9,10,11,None],
[13,None,15,None]])

lista=[12,14,16]
idx_valor = 0

for i, v in enumerate(matrix):
    for j, c in enumerate(v):
        if c is None:
            matrix[i][j] = lista[idx_valor]
            idx_valor+=1
        else:
            pass
matrix

devuelve:
array([[1, 2, 3, 4],
       [5, 6, 7, 8],
       [9, 10, 11, 12],
       [13, 14, 15, 16]], dtype=object)
```

Notar que al crear una variable `idx_valor` que se actualiza solo si se cumplió la condición de valor `None`, permite ir accediendo a los elementos de otro objeto para ser utilizados en el reemplazo.

Ejemplo 3.3.2: resolver el Ejercicio 2.4.3

El código a ejecutar es:

```
productos=['A', 'B', 'C', 'D', 'E', 'F']
precios=[10, 25, 38, 50, 62, 75]
prod_pf = dict(zip(productos, precios))

prod_cli=input('Ingrese los productos comprados separados por coma: ')
prod_cli=prod_cli.split(',')
for i, texto in enumerate(prod_cli):
    prod_cli[i] = texto.upper()

cant_cli=input('Ingrese las cantidades compradas separadas por coma en el mismo orden que los productos: ')
cant_cli=cant_cli.split(',')
cant_comp=[]
for i in cant_cli:
    cant_comp.append(int(i))

prod_cant=dict(zip(prod_cli,cant_comp))

total=[]
```

```

for i in prod_cant:
    for j in prod_pf:
        if i==j:
            total.append(prod_cant[i]*prod_pf[j])

gasto=sum(total)

print('El cliente compró:')
for i in prod_cant:
    for j in prod_pf:
        if i==j:
            print(prod_cant[i], 'unidades de', i, 'a $', prod_pf[j], 'cada
uno')

print('El cliente gastó en total $'+str(gasto))

```

Si se ingresa a, b y luego 2, 4, devuelve:

```

El cliente compró:
2 unidades de A a $ 10 cada uno
4 unidades de B a $ 25 cada uno
El cliente gastó en total $120

```

Nota: En el siguiente [LINK](#) podrá encontrar un cuaderno ejecutable (Sección 3.3 Bucle FOR) donde se muestra cómo operar a través de Python un interador FOR.

3.4. Creación de Funciones

La creación de funciones permite convertir tareas repetitivas en programas generalizados para ser reutilizados una y otra vez (Guttaj, 2017; McKinney, 2018). Esto se logrará a partir de realizar un proceso de abstracción, es decir, se requerirá definir cuáles son los parámetros o argumentos del proceso lógico. A su vez, estos podrán ser fijos o variables. En general, suelen ser variables dado que serán valores ingresados por el usuario requeridos por el código para poder realizar la operación que contenga programada. En este sentido, se trata de la misma lógica abstracta o generalizada involucrada en cualquier función matemática: por cada valor X se obtendrá un nuevo valor Y.

La estructura del código ejecutable será

```

def nombre_función (argumentos):
    """
    Bloque de texto donde se documenta
    """
    bloque de código ejecutable
    return (se deberá especificar el objeto que contiene el resultado)

```

El nombre que llevará la función será elegido por el usuario mientras no sea el de una ya existente en *Python*. Si en la estructura de código anterior, los **argumentos** se encuentran vacíos, implicará que la función actuará con parámetros por default siempre que, el bloque de código este construido para realizar una operación que no depende de valores variables. En la parte que se encuentra entre comillas, se podrá escribir texto para especificar: que hace la función, cuáles son los parámetros y que devuelve. Esta parte será interpretada por *Python* como un no código y por tanto no lo ejecuta. En la parte de bloque de código se deberá codificar las operaciones a ejecutar, cuyo resultado final se deberá almacenar en un objeto. Este objeto será en que luego se debe ubicar seguido al **return** siendo el resultado final que devolverá la ejecución de la función.

Una vez creada la función, sencillamente se deberá invocar su nombre y dentro de los paréntesis, ingresar los valores de los argumentos. Esto significará que la creación de la función se realiza por única vez, almacenándose en memoria, quedando disponible para ser reutilizada las veces que sea necesario durante la sección activa. Los valores de argumentos (parámetros) podrán ser ingresados manualmente o con la función *input()*, razón por la cual puede ser utilizada dentro de un programa interactivo. Finalmente, al ejecutarse, devolverá el resultado esperado.

Ejemplo 3.4.1: cree una función para resolver el **Ejercicio 1.1.2**

El código por ejecutar para crear la función es:

```
def bisiesto(x):  
    '''  
        Función que determina si un año es bisiesto  
        Parámetros:  
            año: número entero correspondiente a un año  
        Devuelve:  
            bisiesto o no bisiesto.  
    '''  
    if not isinstance(x, int):  
        print(f"{x} no es un año válido.")  
        return  
    if x is int and x>=1:  
        resultado="es bisiesto"  
    elif x%400==0:  
        resultado="es bisiesto"  
    elif x%4==0 and x%100!=0:  
        resultado="es bisiesto"  
    else:  
        resultado="no es bisiesto"  
    return resultado
```

Se ejecuta la función, por ejemplo, para 2026:

```
bisiesto(2026)  
devuelve: no es bisiesto
```

Si se ingresa 2024:

```
bisiesto(2024)
```

devuelve: es bisiesto

Si se ingresa 2026.5, devuelve: 2026.5 no es un año válido.

También es posible utilizarla creando un programa, por ejemplo:

```
año=int(input('Ingrese un año: '))
```

```
print(bisiesto(año))
```

si el usuario ingresa 2020, devuelve: es bisiesto

También es posible utilizarla de modo iterativo, por ejemplo:

```
años=[2000, 2010, 2020, 2021, 2022, 2023, 2024, 2025, 2026, 2027,  
2028, 2029, 2030]
```

```
for i in años:
```

```
    print(i, bisiesto(i))
```

devuelve:

2000 es bisiesto

2010 no es bisiesto

2020 es bisiesto

2021 no es bisiesto

2022 no es bisiesto

2023 no es bisiesto

2024 es bisiesto

2025 no es bisiesto

2026 no es bisiesto

2027 no es bisiesto

2028 es bisiesto

2029 no es bisiesto

2030 no es bisiesto

Ejemplo 3.4.2: cree una función para codificar el diagrama del Ejercicio 1.1.3

El código a ejecutar para crear la función es:

```
def mayor(x,y):
```

```
    '''
```

```
        Función que determina si un número es mayor que otro
```

```
        Parámetros:
```

```
            x: primer número
```

```
            y: segundo número
```

```
        Devuelve:
```

```
            cual número es mayor o que se ingresen números distintos si  
            se ingresó números iguales.
```

```
    '''
```

```
    if x!=y:
```

```
        if x>y:
```

```
            resultado= "x="+str(x)+" es mayor que y="+str(y)
```

```
        else:
```

```
            resultado="y="+str(y)+" es mayor que x="+str(x)
```

```

else:
    resultado="debe ingresar dos números distintos"
return resultado

```

Se ejecuta la función, para distintos ejemplos:

```
mayor(3,2)
```

devuelve: x=3 es mayor que y=2

```
mayor(3,3)
```

devuelve: debe ingresar dos números distintos

```
mayor(2,3)
```

devuelve: y=3 es mayor que x=2

Ejemplo 3.4.3: cree una función para codificar el **Ejercicio 1.1.4**

El código a ejecutar para crear la función es:

```

def sumar():
    """
        Función que suma una serie del 1 al 10 inclusive
        Parámetros:
            ninguno
        Devuelve:
            la suma del 1 al 10.
    """
    num=0
    suma=0
    while num<11:
        suma=suma+num
        num+=1
    return suma

```

Se ejecuta la función:

```
sumar()
```

devuelve: 55

Ejercicio 3.4.4: cree una función que resuelva el **Ejercicio 2.4.2**

El código a ejecutar para crear la función es:

```

def datos_personales(nombre, apellido, nacionalidad, edad):
    """
        Función que devuleve datos personales
        Parámetros:
            nombre: nombre ingresado por el usuario
            apellido: apellido ingresado por el usuario
            nacionalidad: nacionalidad ingresada por el usuario
            edad: edad ingresada por el usuario
        Devuelve:
            El ciudadano (nombre y apellido) de nacionalidad
            (nacionalidad) nació en el año (año nacimiento).
    """

```



```
'''

nombre=nombre.upper()
apellido=apellido.upper()
nacionalidad=nacionalidad.upper()
import datetime
hoy=datetime.date.today()
año_actual=hoy.year
año_nacimiento=año_actual-int(edad)
resultado='El ciudadano ' + str(nombre) + ' ' + str(apellido) + '
de nacionalidad ' + str(nacionalidad) + ' nació en el año
'+str(año_nacimiento)
return resultado
```

Se ejecuta la función:

```
datos_personales(input('ingrese su nombre: '),
                  input('ingrese su apellido: '),
                  input('ingrese su nacionalidad: '),
                  input('ingrese su edad: '))
```

devuelve: ingrese su nombre: juan
ingrese su apellido: perez
ingrese su nacionalidad: argentino
ingrese su edad: 40
El ciudadano JUAN PEREZ de nacionalidad ARGENTINO nació en el año 1986

Ejercicio 3.4.5: cree una función que resuelva el Ejercicio 2.4.4

El código a ejecutar para crear la función es:

```
def markov_esperado(vector1, vector2, vector3, vector4, vector5,
vector6):
'''
    Función que devuelve el monto total esperado que sea pagado y
    el monto total que será incobrable
    Parámetros:
        vector1: primera fila de la matriz de transición
        vector2: segunda fila de la matriz de transición
        vector3: tercera fila de la matriz de transición
        vector4: cuarta fila de la matriz de transición
        vector5: quinta fila de la matriz de transición
        vector6: vector inicial
    Devuelve:
        Se espera que (MONTO PAGADOS) sean pagados y que (MONTO
        INCOBRABLE) sean incobrables.
'''

import numpy as np
matriz=np.array([vector1, vector2, vector3,vector4,vector5])
matriz_N=matriz[0:3,0:3]
```

```

matriz_I=np.array([[1, 0, 0], [0, 1, 0], [0, 0, 1]])
inv=matriz_I-matriz_N
inversa=np.linalg.inv(inv)
matriz_A=matriz[0:3,3:6]
matriz_PA=inversa @ matriz_A
vector6=np.array([vector6])
vector_esp=np.dot(vector6,matriz_PA)
vector_esp=np.round(vector_esp,0)
resultado='Se espera que $'+str(vector_esp[0][0]) + ' sean
pagados y que $' + str(vector_esp[0][1]) + ' sean incobrables'
return resultado

```

Se ejecuta la función:

```

markov_esperado([0.3, 0.2, 0.1, 0.4, 0.0],
                [0.3,0.3,0.1,0.2,0.1],
                [0,0,0.4,0.5,0.1],
                [0,0,0,1,0],
                [0,0,0,0,1],
                [150000,60000,25000])

```

devuelve: Se espera que \$206531.0 sean pagados y que \$28469.0 sean incobrables

Nota: observar que esta función solo será válida para una matriz de transición de 5x5

Nota: En el siguiente [LINK](#) podrá encontrar un cuaderno ejecutable (Sección 3.4 Creación de Funciones) donde se muestra cómo crear funciones a través de Python.

3.5. Ejercicios propuestos

Ejercicio 3.5.1: Cree una función para ser utilizada en un programa donde a partir de que el usuario ingrese su email, devuelva a que proveedor (por ejemplo, Gmail pertenece a Google) pertenece su email o en su defecto 'No identificado', independientemente de si está escrito con mayúscula o minúscula.

Ejercicio 3.5.2: Cree una función para ser utilizada en un programa, a partir de que el usuario ingrese una secuencia de números y alguno vacío, que devuelva una lista con todos los valores y los vacíos imputados con cero

Ejercicio 3.5.3: Cree una función para ser utilizada en un programa donde a partir de que el usuario ingrese código de producto y cantidad separada por coma como pedido sobre el menú existente, devuelva el detalle del pedido y el total a pagar

Menu existente:

Empanada de carne \$500 c/u
 Empanada de jamón y queso \$500 c/u
 Empanada de humita \$500 c/u
 Pizza de muzzarella \$15000 c/u
 Pizza de napolitana \$18000 c/u
 Pizza de fugazetta \$17000 c/u

Ejercicio 3.5.4: Cree una función para ser utilizada en un programa donde a partir de que el usuario ingrese:

- las cantidades y precios consumidas el mes anterior
- las cantidades y precios consumidas el mes actual

devuelva el valor del Índice de Laspeyre (IL), Paschee (IP) y Fisher (IF), donde cada uno son:

$$IL_t = \frac{\sum_{i=1}^n q_{i,0} * p_{i,t}}{\sum_{i=1}^n q_{i,0} * p_{i,0}} \quad IP_t = \frac{\sum_{i=1}^n q_{i,t} * p_{i,t}}{\sum_{i=1}^n q_{i,t} * p_{i,0}} \quad IF_t = \sqrt{IL_t * IP_t}$$

4. Bibliografía

Alpha C. Chiang, & Wainwright, K. (2006). Métodos fundamentales de economía matemática. McGraw-Hill.

Guttag, J. (2017) Introduction to Computation and Programming Using Python: With Application to Understanding Data. Cambridge MIT Press.

McKinney, W. (2018). Python For Data Analysis. O ´ Reilly Media Inc.

Ross, S. M. (2014). Introduction to probability models. Academic press.

5. Anexo

Acceso al repositorio completo con diferentes cuadernos Colaboratory que acompañan este libro: [LINK DE ACCESO](#)